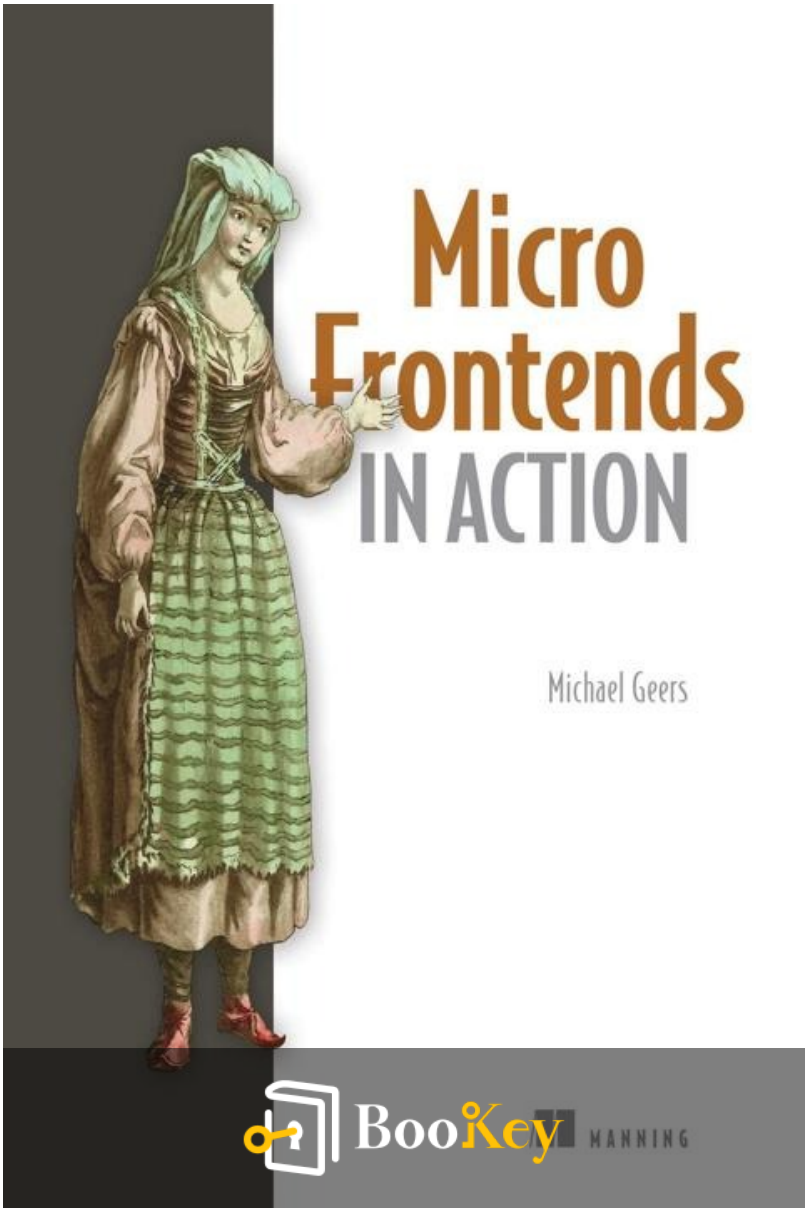
Micro Frontends in Action PDF

Michael Geers



More Free Book
Scan to Download
Listen It

Micro Frontends in Action

Transforming Frontend Development Through Micro Frontends Architecture.

Written by Bookey

Check more about Micro Frontends in Action Summary

Listen Micro Frontends in Action Audiobook

More Free Book

Scan to Download

Listen It

About the book

Micro Frontends in Action offers a comprehensive guide to adopting the microservices paradigm for frontend development, allowing you to tackle the complexities of modern web applications. As browser-based software evolves, it often transitions into tangled single-page applications that are tough to manage. This book advocates for a micro frontends approach, enabling you to build web applications as collections of well-defined, stand-alone components. Through hands-on projects, including the development of an e-commerce application, you'll gain insights into essential practices like server-side and client-side composition, routing, and maintaining visual consistency. Geared towards web developers, software architects, and team leaders, this guide equips you with the knowledge to enhance feature delivery speed, streamline upgrades, and choose diverse technologies that best fit your stack. With practical strategies for independent component development and effective team workflows, Micro Frontends in Action is your essential resource for modern frontend architecture.

More Free Book

Scan to Download

Listen It

About the author

Michael Geers is a seasoned technology consultant and architect known for his expertise in modern web development, particularly in the realm of micro frontends. With a strong background in software engineering and a passion for creating scalable, maintainable applications, Geers has actively contributed to the discourse around frontend architecture through his speaking engagements, workshops, and writings. His insights, drawn from extensive experience in various industries, provide valuable perspectives on how to effectively implement micro frontend strategies, enabling teams to deliver cohesive user experiences while maintaining modularity and flexibility. Geers' thought leadership in this emerging field positions him as a key figure in guiding developers and organizations toward more innovative and efficient approaches to web application development.

More Free Book

Scan to Download

Listen It

Ad

Scan to Download

Try Bookey App to read 1000+ summary of world best books

Unlock 1000+ Titles, 80+ Topics

New titles added every week

Brand | Leadership & Collaboration | Time Management | Relationship & Communication

ness Strategy | Creativity | Public | Money & Investing | Know Yourself | Positive P

Entrepreneurship | World History | Parent-Child Communication | Self-care | Mind & Spi

Insights of world best books

Free Trial with Bookey

Summary Content List

Chapter 1 : Micro Frontends in Action

Chapter 2 : 1 What are micro frontends?

Chapter 3 : 2 My first micro frontends project

Chapter 4 : Part 2. Routing, composition, and communication

Chapter 5 : 3 Composition with Ajax and server-side routing

Chapter 6 : 4 Server-side composition

Chapter 7 : 5 Client-side composition

Chapter 8 : 6 Communication patterns

Chapter 9 : 7 Client-side routing and the application shell

Chapter 10 : 8 Composition and universal rendering

Chapter 11 : 9 Which architecture fits my project?

Chapter 12 : Part 3. How to be fast, consistent, and effective

Chapter 13 : 10 Asset loading

Chapter 14 : 11 Performance is key

Chapter 15 : 12 User interface and design system

More Free Book

Scan to Download

Listen It

Chapter 16 : 13 Teams and boundaries

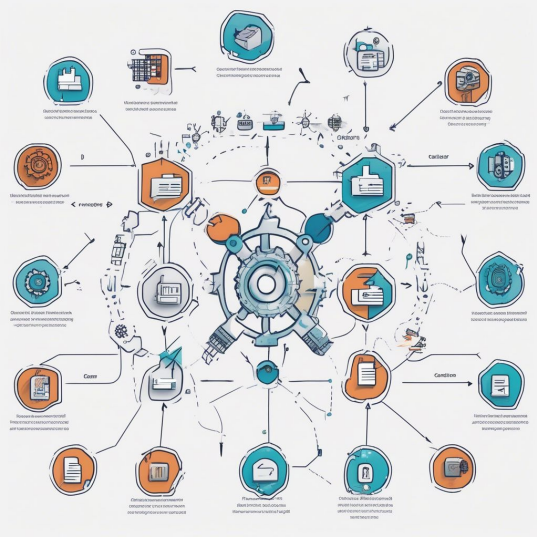
Chapter 17 : 14 Migration, local development, and testing

More Free Book

Scan to Download

Listen It

Chapter 1 Summary : Micro Frontends in Action



Micro Frontends in Action - Summary of Chapter 1

Introduction to Micro Frontends

Michael Geers shares over 20 years of experience in web application development, highlighting the evolution and organizational needs of different-sized projects. He recalls a significant project in 2014 focused on rebuilding a monolithic e-commerce system plagued by performance issues and slow feature development.

More Free Book
Scan to Download
Listen It

The Challenge of Monolithic Architectures

In larger projects, adding new features often led to unintended breaks in unrelated system parts. Geers emphasizes the need for an architecture that supports independent, parallel feature development among teams.

Verticalization Approach

The solution adopted was called "verticalization," a method where cross-functional teams independently build and evolve specific areas of the application, contributing to a more manageable and flexible e-commerce system.

Origins of Micro Frontends

Though various companies were experimenting with similar architectures, the term "micro frontends" was introduced by the ThoughtWorks Technology Radar in November 2016, which helped formalize the concept and foster community dialogue.

Writing the Book

More Free Book
Scan to Download
Listen It

Inspired by the growing interest and demand for knowledge sharing in this architecture, Geers took on the challenge of writing this book, aiming to document and explain the motivations, concepts, and practical techniques of implementing micro frontends.

Acknowledgments

Geers acknowledges the contributions of friends, family, colleagues, and the Manning team in the book creation process, stressing the collaborative nature of the effort.

About This Book

"Micro Frontends in Action" addresses the fundamental principles of micro frontends rather than focusing on specific tools or libraries. It covers practical integration techniques while emphasizing adherence to web standards. It caters to a diverse audience, including frontend developers, backend developers, and software architects who have a foundation in web technologies.

Book Structure

More Free Book
Scan to Download
Listen It

The book is divided into three parts and includes 14 chapters focusing on:

1. Understanding what micro frontends are and their use cases.
2. Technical approaches for frontend integration from different teams.
3. Best practices for performance and design consistency across distributed teams.

Through this structured approach, readers will gain insights into the benefits and potential challenges of adopting micro frontends, preparing them for practical application in real-world projects.

More Free Book

Scan to Download

Listen It

Example

Key Point:Adopting micro frontends allows independent team collaboration, leading to faster feature development.

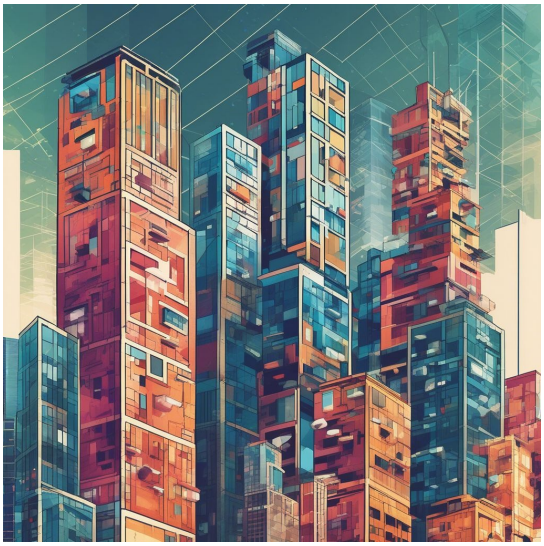
Example:Imagine you are working on a large e-commerce platform, and your team is responsible for the checkout process. Instead of waiting for other teams to finish their parts of the monolithic system, you can develop and deploy your checkout feature independently. This independence means you can enhance usability and performance without risking disruption to other features like product listings or user accounts, ultimately accelerating your delivery speed and improving user satisfaction.

More Free Book

Scan to Download

Listen It

Chapter 2 Summary : 1 What are micro frontends?



Section	Content
1. What are micro frontends?	- Definition and significance of micro frontends- Comparison with other architectures- Importance of scaling frontend development- Challenges introduced by this architecture
1.1 The big picture	- Autonomy of vertically organized software systems- Teams manage their own domains with integration techniques for assembly
1.1.1 Systems and teams	- Autonomous teams with their data stores- Minimal reliance on synchronous calls- Independent generation and maintenance of user interfaces
1.1.2 The frontend	- End-to-end responsibility for functionality- Independent technology stack choice and dependency upgrades
1.1.3 Frontend integration	- Integration techniques: routing, composition, communication- Server-side and client-side methods
1.1.4 Shared topics	- Importance of web performance, design systems, and knowledge sharing
1.2 What problems do micro frontends solve?	- Optimizing feature development timelines- Eliminating frontend monolith concerns- Accommodating continuous technology evolution
1.2.1 Optimize for feature development	- Enhanced development speed through reduced inter-team coordination
1.2.2 No more frontend monolith	- Decentralized code base for complexity reduction and maintainability
1.2.3 Be able to keep changing	- Swift adaptation to new technologies
1.2.4 The benefits of independence	- Innovation without being disrupted by dependencies
1.3 The downsides of micro frontends	- Challenges related to redundancy, consistency, heterogeneity, and code complexity

More Free Book
Scan to Download
Listen It

Section	Content
1.4 When do micro frontends make sense?	- Effective for medium-to-large projects- Not beneficial for smaller projects due to complexity
1.4.1 Good for medium-to-large projects	- Autonomy and reduced communication enhance scalability
1.4.2 Works best on the web	- Flexibility of the web platform enhances micro frontends
1.4.3 Productivity versus overhead	- Dividing applications presents organizational and technical complexities
1.4.4 Where micro frontends are not a great fit	- Not suitable for smaller teams or efficiently communicating projects
1.4.5 Who uses micro frontends?	- Companies like Amazon, IKEA, and Zalando enhance development processes with micro frontends
Summary	- Structure for integrating independent teams for feature delivery- Useful for larger projects focusing on rapid iterations- Presents organizational challenges

1. What are micro frontends?

This chapter covers:
- Discovering what micro frontends are
- Comparing the micro frontends approach to other architectures
- Pointing out the importance of scaling frontend development
- Recognizing the challenges that this architecture introduces

Micro frontends provide an alternative approach to frontend architecture by splitting applications into vertical slices, each managed by a dedicated team responsible for the entire stack from database to user interface. This approach optimizes

More Free Book
Scan to Download
Listen It

development speed, eases frontend upgrades, and increases customer focus.

1.1 The big picture

- Micro frontends are driven by the autonomous functionality of vertically organized software systems, allowing teams to operate independently.
- Each team is responsible for its domain, and integration techniques come into play to assemble components.

1.1.1 Systems and teams

- Teams operate autonomously with their data stores and minimal reliance on synchronous calls.
- Ownership of user interfaces allows teams to generate and maintain features independently.

1.1.2 The frontend

- Teams have end-to-end responsibility for delivering their functionality, developing user interfaces as micro frontends.
- Teams can independently select technology stacks and upgrade dependencies.

More Free Book
Scan to Download
Listen It

1.1.3 Frontend integration

- Techniques for frontend integration fall into routing, composition, and communication.
- Integration methods can be server-side or client-side, affecting how components interact and communicate.

1.1.4 Shared topics

- Shared concerns like web performance, design systems, and knowledge sharing are critical for cohesive operation among autonomous teams.

1.2 What problems do micro frontends solve?

- Micro frontends aim to optimize feature development timelines, eliminate frontend monolith concerns, and accommodate continuous evolution of technology and processes.

1.2.1 Optimize for feature development

- Reduction of inter-team coordination improves

More Free Book
Scan to Download
Listen It

development speed and allows agile iterations.

1.2.2 No more frontend monolith

- Micro frontends decentralize the frontend code base, reducing complexity and enhancing maintainability.

1.2.3 Be able to keep changing

- Teams can swiftly adapt and adopt new technologies, which is necessary in the evolving field of frontend development.

1.2.4 The benefits of independence

- Autonomy allows teams to innovate without being disrupted by dependencies on other teams.

1.3 The downsides of micro frontends

- With autonomy comes challenges like redundancy, consistency, heterogeneity, and increased code complexity in a micro frontends architecture.

1.4 When do micro frontends make sense?

More Free Book
Scan to Download
Listen It

- Effective for medium-to-large projects, micro frontends reduce communication overhead as team sizes grow. However, smaller projects may not benefit from this architecture due to the additional complexities it introduces.

1.4.1 Good for medium-to-large projects

- The autonomy and reduced communication barrier enhance scalability for larger teams.

1.4.2 Works best on the web

- Micro frontends excel in web applications due to the inherent flexibility of the web platform.

1.4.3 Productivity versus overhead

- While dividing applications into smaller systems yields benefits, it introduces organizational and technical complexities.

1.4.4 Where micro frontends are not a great fit

More Free Book
Scan to Download
Listen It

- Micro frontends may not be suitable for smaller teams or projects where communication is already efficient.

1.4.5 Who uses micro frontends?

- Various companies, especially in e-commerce, like Amazon, IKEA, and Zalando, have successfully implemented micro frontends to enhance their development processes.

Summary

Micro frontends provide a structure for integrating independent teams focused on delivering specific features while handling the complexities of frontend development. They are particularly useful for larger projects focusing on rapid iterations and continuous improvements, albeit presenting certain organizational challenges.

More Free Book
Scan to Download
Listen It

Critical Thinking

Key Point:Autonomy and independent team operation in micro frontend architecture

Critical Interpretation:While the concept of micro frontends emphasizes the benefits of autonomy and independence for teams working on different parts of a project, one must critically assess whether such a divisive structure truly promotes innovation or merely fosters silos of knowledge that can lead to redundancy and inconsistency. The author's viewpoint may be colored by practical successes observed in larger enterprises; however, research indicates that excessive autonomy can hinder effective communication across teams, potentially resulting in fragmented systems. As noted in studies on team dynamics (Bion, 1961; Duhigg, 2016), collaboration often enhances shared understanding and efficiency, suggesting that a balance between autonomy and interdependence may yield superior outcomes. Furthermore, organizations like Spotify and Atlassian exemplify the potential drawbacks of overspecialization in collaborative projects, where integration challenges spring from distinct team operations.

More Free Book
Scan to Download
Listen It

Chapter 3 Summary : 2 My first micro frontends project

Section	Content
Chapter Title	My First Micro Frontends Project
Overview	Covers building a micro frontends application, linking pages, and integrating fragments via iframes.
Key Feature	Enables multiple teams to work in parallel on complex applications with minimal coupling.
Example Project	The Tractor Store, an e-commerce site for toy tractor models created by Tractor Models, Inc.
Teams Involved	Team Decide (product detail page) and Team Inspire (recommendations).
Initial Setup	Team Decide uses MongoDB and Node.js; Team Inspire uses Python for data science.
Integration Focus	Integration through links and iframes with a strong emphasis on independent deployments.
Page Linking	Teams create standalone pages and define URL patterns to link product and recommendation pages.
Code Access	Code for integration is stored in designated folders, with resources for running examples.
Conclusion	Sets the foundation for understanding micro frontends and team collaboration.

My First Micro Frontends Project

This chapter covers:
- Building the micro frontends example application for this book
- Connecting pages from two teams via links
- Integrating a fragment into a page via iframes

The essential feature of micro frontends is enabling multiple teams to work in parallel on complex applications. However, the end user is indifferent to the internal team structure.

More Free Book
Scan to Download
Listen It

Thus, integrating user interfaces created by different teams becomes crucial. In this chapter, you will learn integration techniques using links and iframes. While these technologies might not be new or exciting, they are easy to implement and understand, allowing for minimal coupling between teams. This loose coupling fosters maximum freedom for teams to focus on their specific missions.

Throughout the book, we will develop an example project called "The Tractor Store," expanding its complexity as we explore various integration techniques, each with its advantages and disadvantages. There is no single "gold standard" for integration; the best approach depends on the specific use case. The chapter begins with simple scenarios before progressing to more complex integrations.

Introducing The Tractor Store

Tractor Models, Inc. is an imaginary company creating

Install Bookey App to Unlock Full Text and Audio

More Free Book

Scan to Download

Listen It

Ad

Scan to Download

Why Bookey is must have App for Book Lovers

30min Content
The deeper and clearer interpretation we provide, the better grasp of each title you have.

Text and Audio format
Absorb knowledge even in fragmented time.

Quiz
Check whether you have mastered what you just learned.

And more
Multiple Voices & fonts, Mind Map, Quotes, IdeaClips...

Free Trial with Bookey

Chapter 4 Summary : Part 2. Routing, composition, and communication

Part 2: Routing, Composition, and Communication

In this section, we'll delve deeper into the techniques necessary for building advanced micro frontends projects. We won't rely on specialized tools since the adoption of micro frontends is broadening within the software industry, leading to an increase in the availability of meta-frameworks and helper libraries. These resources often address common challenges and enhance the developer experience; however, we will only briefly touch on them as the ecosystem continues to evolve.

Core Principles and Approach

Our focus will remain on existing web standards and leveraging native browser features to maintain stability and value, as proven in past projects. A strong understanding of core concepts is essential for successfully building projects, even if you later choose to integrate a micro frontends

More Free Book
Scan to Download
Listen It

library.

Chapter Overview

Chapters 3 to 8 will cover techniques for routing, composition, and communication within both server-side and client-side rendered web applications, arranged by increasing complexity. We will start with simpler techniques and progressively introduce more sophisticated ones. Chapter 9 will provide an architectural overview, contextualizing the learned techniques and aiding in making informed architectural decisions for future projects.

More Free Book
Scan to Download
Listen It

Chapter 5 Summary : 3 Composition with Ajax and server-side routing

Section	Summary
Chapter Title	Composition with Ajax and Server-side Routing
Key Topics	Integrating fragments into a page via Ajax Applying project-wide namespaces to avoid style or script collisions Utilizing the Nginx web server to serve all applications from one domain Implementing request routing to forward incoming requests to the right server
Previous Chapter Focus	UI integration using links and iframes, highlighting isolation but with usability and SEO drawbacks.
Ajax Integration Purpose	To enhance SEO and user experience by replacing iframes with Ajax for deeper integration.
Team Responsibilities	Team Inspire: Develops HTML fragments and styles. Team Decide: Implements the Ajax call to fetch and integrate fragments into the DOM.
Implementation Steps	Create fragment endpoints accessible via specified URLs. Develop HTML and CSS for fragments. Use Fetch API to load fragments into the product page. Ensure smooth integration and collaboration between teams.

Composition with Ajax and Server-side Routing

This chapter covers:

- Integrating fragments into a page via Ajax
- Applying project-wide namespaces to avoid style or script collisions

More Free Book
Scan to Download
Listen It

- Utilizing the Nginx web server to serve all applications from one domain
- Implementing request routing to forward incoming requests to the right server

The previous chapter prepared applications for two teams, focusing on user interface integration through links and iframes. While these methods offer strong isolation, they present drawbacks concerning usability, performance, layout flexibility, accessibility, and search engine optimization. This chapter addresses these issues by exploring Ajax for fragment integration and configuring a shared web server.

3.1 Composition via Ajax

Customer feedback indicates improved engagement on the new product page. However, concerns arise regarding poor search engine rankings, attributed to iframe usage. To enhance SEO, the teams decide to transition from iframes to Ajax for deeper integration. Team Inspire will deliver recommendations as HTML fragments, which Team Decide will load and integrate into the product page's DOM.

3.1.1 How to Do It

More Free Book
Scan to Download
Listen It

Creating the fragment endpoint is straightforward. Team Inspire needs to set up a new endpoint for the fragment markup, which is accessible via a specified URL. Both teams can implement their changes in parallel, allowing for productive collaboration.
Team Inspire develops an HTML file for each fragment, which contains streamlined versions of the recommendation page. They also create dedicated styles, while Team Decide prepares a script that triggers the Ajax call.

Markup and Ajax Request

The fragment markup references its own CSS file, which is crucial for styling when loaded into the product page. Team Decide, through client-side JavaScript, can fetch the fragment HTML via the Fetch API and append it to the DOM, ensuring a smooth integration process.
In summary, this chapter outlines the transition to Ajax-based fragment integration to improve both user experience and search engine ranking, while detailing the implementation steps necessary for successful collaboration between two teams.

More Free Book
Scan to Download
Listen It

Chapter 6 Summary : 4 Server-side composition

Server-side Composition

This chapter covers:
- Examining server-side composition using Nginx and SSI.
- Investigating how timeouts and fallbacks can help when dealing with broken or slow fragments.
- Comparing the performance characteristics of different composition techniques.
- Exploring alternative solutions like Tailor, Podium, and ESI.

In previous chapters, techniques such as client-side integration were discussed. This chapter builds upon those, focusing on server-side integrations. Server-side composition assembles various fragments into a complete page on the server before sending it to the client.

4.1 Composition via Nginx and Server-Side Includes (SSI)

More Free Book
Scan to Download
Listen It

The team at Tractor Models, Inc. transitioned from Ajax-based integration to using server-side integration. They found that server-side integrations led to better performance and faster load times. By using Nginx's SSI, they can include references to asset URLs directly in the HTML response, leading to quicker load times.

4.1.1 How to do it

The SSI technique has historical significance, dating back to the 1990s. Nginx’s configuration allows teams to enable SSI and include fragments efficiently. Once SSI support is activated and the include directive is placed, the server handles integration seamlessly.

4.1.2 Better Load Times

Testing under simulated 3G conditions, the server-side

Install Bookey App to Unlock Full Text and Audio

More Free Book

Scan to Download

Listen It

Ad

Scan to Download

App Store
Editors' Choice

22k 5 star review

Positive feedback

Sara Scholz

tes after each book summary
erstanding but also make the
and engaging. Bookey has
ding for me.

Fantastic!!!

Masood El Toure

I'm amazed by the variety of books and languages Bookey supports. It's not just an app, it's a gateway to global knowledge. Plus, earning points for charity is a big plus!

Fi
Ab
bo
to
my

José Botín

ding habit
p's design
ual growth

Love it!

Wonnie Tappkx

Bookey offers me time to go through the important parts of a book. It also gives me enough idea whether or not I should purchase the whole book version or not! It is easy to use!

Time saver!

Bookey is my go-to app for
summaries are concise, ins
curated. It's like having acc
right at my fingertips!

Awesome app!

Rahul Malviya

I love audiobooks but don't always have time to listen to the entire book! bookey allows me to get a summary of the highlights of the book I'm interested in!!! What a great concept !!!highly recommended!

Beautiful App

Alex Walk

This app is a lifesaver for book lovers with busy schedules. The summaries are spot on, and the mind maps help reinforce wh I've learned. Highly recommend!

Free Trial with Bookey

Chapter 7 Summary : 5 Client-side composition

Client-side Composition

This chapter discusses client-side integration techniques, focusing on Web Components as a method for micro frontend development. It explores the integration of micro frontends built with different frameworks on the same page and how Shadow DOM can help avoid style conflicts when introducing a micro frontend into legacy systems.

Introduction to Client-side Rendering

Traditional architectures rely on monolithic frontends tied to a specific framework and version, leading to complex upgrade cycles. In contrast, micro frontends allow for independently upgradable user interfaces that can coexist on the same page, leveraging Web Components for seamless integration.

Wrapping Micro Frontends in Web Components

More Free Book
Scan to Download
Listen It

A case study of Tractor Models, Inc. illustrates how a third team (Team Checkout) created a Buy button as a standalone micro frontend. This button is implemented as a Web Component that can be easily integrated into product detail pages by other teams without needing to understand its internal workings.

Web Components and Custom Elements

Web Components are standardized APIs that allow for greater encapsulation and interoperability. The Custom Elements feature enables developers to create reusable components that can function similarly to HTML elements. An example Buy button shows how encapsulating logic and interactions allows for easier integration and maintenance.

Style Isolation with Shadow DOM

Shadow DOM offers a way to isolate styles within a component, preventing style leakage and conflicts between micro frontends. It allows for robust applications by encapsulating styles and markup so they do not affect the broader page.

More Free Book
Scan to Download
Listen It

Creating and Using Shadow DOM

Developers can create an isolated subtree by calling `.attachShadow()` on an element. Two modes (open and closed) determine the accessibility of the shadow root, affecting how external scripts can interact with it.

Advantages and Disadvantages of Web Components

The chapter discusses the benefits, such as standardization and encapsulation, as well as drawbacks like the requirement for client-side JavaScript and challenges with older browser compatibility.

Integration Use Cases

Choosing to use Web Components as a method for client-side integration is particularly effective for interactive applications. However, for simpler sites, server-side rendering may still be a better option.

Summary

More Free Book
Scan to Download
Listen It

Web Components allow developers to encapsulate micro frontend applications and provide a standard way for teams to interact with and integrate these components. The integration of Shadow DOM further reinforces style isolation, making it suitable for combining different team UIs without conflicts. This chapter highlights the balance between client-side and server-side rendering techniques in developing robust applications.

More Free Book
Scan to Download
Listen It

Chapter 8 Summary : 6 Communication patterns

Communication Patterns

This chapter covers the essential aspects of communication between micro frontends, including user interface (UI) communication patterns, state management, and server communication organization.

6.1 User Interface Communication

The necessity for UI components from different teams to communicate is explored, particularly in scenarios where users interact across various elements. For example, when a user adds an item to their basket, other micro frontends need updates to reflect this action.

6.1.1 Parent to Fragment Communication

An example illustrates how selections made on a product page (e.g., selecting a platinum edition tractor) need to be

More Free Book
Scan to Download
Listen It

propagated down to a "Buy" button component. The communication is implemented through attributes and accompanying JavaScript to update the relevant components in response to user actions.

6.1.2 Fragment to Parent Communication

This section discusses how Team Checkout’s "Buy" button can emit events to notify Team Decide of user interactions (e.g., the successful addition of an item to the cart). This communication is performed using the native CustomEvents API, promoting a clear contract between the interacting components.

6.1.3 Fragment to Fragment Communication

In scenarios where one fragment needs to notify another (for instance, when an item is added to the cart and needs to be displayed in a mini-cart), communication can be done through a global event bus. This approach enhances decoupling through the use of events, allowing fragments to notify each other without direct dependencies.

Important Considerations

More Free Book
Scan to Download
Listen It

-

State Management

: Care should be taken with shared states and context to prevent duplication and maintain consistency across micro frontends.

-

Event Bubbling

: Custom events should enable bubbling to allow components to listen for events at a higher level, further promoting decoupling.

-

Payload Minimization

: While passing data with events (like SKU or edition), keep payloads minimal to maintain loose coupling. Events are primarily for notifications rather than extensive data transfers.

Overall, the chapter emphasizes the importance of various communication patterns between micro frontends to ensure cohesive user experiences while maintaining autonomy and reducing dependencies among teams.

More Free Book

Scan to Download

Listen It

Example

Key Point:Communication patterns are vital in micro frontends for seamless user interactions.

Example:Imagine you are shopping for a new tractor online. As you browse the product details, you select the platinum edition and click ‘Add to Cart.’ Instantly, the related item updates in the mini-cart at the corner of your screen, acknowledging your choice without needing to refresh the page. This smooth interaction happens due to effective communication between the different micro frontends, allowing each to remain independent yet responsive to your actions, ensuring you have a cohesive and enjoyable shopping experience.

More Free Book

Scan to Download

Listen It

Chapter 9 Summary : 7 Client-side routing and the application shell

Client-side Routing and the Application Shell

Overview

This chapter explores client-side routing concepts applied to single-page applications (SPAs) and the construction of a shared application shell as a unified entry point for users. It compares different routing techniques and discusses the micro frontends meta-framework, single-spa, which simplifies integration.

Key Concepts

-
Inter-team Routing
: Building on communication techniques from earlier chapters, this section focuses on page-level integration for micro frontends.

More Free Book
Scan to Download
Listen It

-

SPAs vs Monolithic Applications

: A distinction between hard navigation (full page refresh) and soft navigation (client-side rendered transitions) is made, highlighting their impacts on user experience.

-

Integration Approaches

: The chapter presents two main models for page transitions -

-

Links Only Model

: Utilizes plain links resulting in hard navigations between team boundaries.

-

Linked Single-Page Apps

: Allows soft navigation within team boundaries, switching to hard navigation at team borders.

Install Bookey App to Unlock Full Text and Audio

More Free Book

Scan to Download

Listen It

Ad

Scan to Download

Read, Share, Empower

Finish Your Reading Challenge, Donate Books to African Children.

The Concept

BOOKS FOR AFRICA × ×

This book donation activity is rolling out together with Books For Africa. We release this project because we share the same belief as BFA: For many children in Africa, the gift of books truly is a gift of hope.

The Rule

Earn 100 points → **Redeem a book** → **Donate to Africa**

Your learning not only brings knowledge but also allows you to earn points for charitable causes! For every 100 points you earn, a book will be donated to Africa.

Free Trial with Bookey

Chapter 10 Summary : 8 Composition and universal rendering

Section	Summary
Chapter Title	Composition and Universal Rendering
Terminology	Universal rendering, Isomorphic JavaScript, and SSR refer to the use of a single codebase for rendering on both server and client sides, with focus on "universal rendering."
8.1 Combining Server- and Client-Side Composition	A hybrid model was adopted to address sales drop, ensuring quick loads and interactivity through both server-side and client-side composition.
8.1.1 SSI and Web Components	Server-side includes (SSI) prepopulate Web Components, enhancing user interface resilience and enabling progressive enhancement.
8.1.2 Contract Between the Teams	Clear integration contracts are essential, defining client-side components and server-side endpoints, with standardized naming to ease integration.
8.1.3 Other Solutions	Beyond SSI and Web Components, other techniques like ESI and frameworks such as Ara support universal rendering and client-side hydration.
8.2 When Does Universal Composition Make Sense?	Essential for applications needing fast load times and interactivity, though it adds complexity due to the integration of both client and server techniques.
8.2.1 Universal Rendering with Pure Server-Side Composition	Pure server-side composition is adequate when interactivity is not needed, simplifying architecture.
8.2.2 Increased Complexity	Universal composition improves user experience but increases complexity in setup and skills requirements for developers.
8.2.3 Universal Unified Single-Page App?	While complex, combining an application shell model with universal rendering is possible with evolving development frameworks.
Summary	Universal rendering leverages the strengths of both server and client rendering for fast, responsive interfaces, necessitating close team collaboration and architectural rigor despite integration challenges.

8 Composition and Universal Rendering

This chapter covers the integration of universal rendering within a micro frontends architecture, employing both server-side and client-side composition techniques for

More Free Book
Scan to Download
Listen It

optimal performance and user experience.

Terminology: Universal, Isomorphic, and SSR

Universal rendering, Isomorphic JavaScript, and server-side rendering (SSR) refer to rendering markup using a single codebase on both the server and client. This chapter emphasizes the term "universal rendering."

8.1 Combining Server- and Client-Side Composition

Amid increasing demand for a product, the development team faced quality issues after a client-side JavaScript error led to a 34% drop in sales. To remedy this, they adopted a hybrid integration model using server-side composition alongside existing client-side technologies. This approach enables quick initial loads while still allowing for interactive client-side updates.

8.1.1 SSI and Web Components

Using server-side includes (SSI), the team enhances the Buy button's Web Component by prepopulating server-side markup. The combination of client and server composition

More Free Book
Scan to Download
Listen It

leads to a more resilient and user-friendly interface, enabling progressive enhancement.

8.1.2 Contract Between the Teams

Teams must establish a clear contract for integration, providing both the client-side component definitions and the server-side markup endpoints. A standardized naming schema can streamline integration.

8.1.3 Other Solutions

Various integration techniques exist beyond SSI and Web Components, including ESI and using frameworks like Ara that focus on universal rendering and client-side hydration.

8.2 When Does Universal Composition Make Sense?

Universal composition is necessary for applications requiring fast load times, high interactivity, and seamless communication between micro frontends. It can combine client-side and server-side techniques, although it introduces added complexity.

More Free Book
Scan to Download
Listen It

8.2.1 Universal Rendering with Pure Server-Side Composition

In cases where interactivity between micro frontends is unnecessary, pure server-side composition suffices, simplifying the architecture.

8.2.2 Increased Complexity

While universal composition offers significant advantages in user experience, it incurs complexity in setup, debugging, and onboarding. Developers must be adept in both server integration and client hydration.

8.2.3 Universal Unified Single-Page App?

Combining an application shell model with universal rendering is complex, though theoretically possible. Development frameworks are evolving to support such capabilities.

Summary

Universal rendering merges the advantages of both server

More Free Book
Scan to Download
Listen It

and client rendering, ensuring fast loads and responsive user interfaces. Implementing this requires close collaboration between teams to deliver micro frontends efficiently while adhering to robust architectural standards. While the integration complexity is notable, the benefits—particularly user experience—are substantial.

More Free Book
Scan to Download
Listen It

Chapter 11 Summary : 9 Which architecture fits my project?

Which architecture fits my project?

This chapter discusses the options for micro frontend architectures, comparing their benefits and challenges, and helping to determine the best fit for specific project needs. It revisits integration techniques previously covered, highlights terminology, and introduces the Documents-to-Applications Continuum to guide architectural decisions. The chapter concludes with a decision-making guide.

9.1 Revisiting the terminology

Establishing a common vocabulary across teams is essential when setting up a micro frontends project. The techniques are categorized into two groups: routing and page transition, and composition techniques.

9.1.1 Routing and page transitions

More Free Book
Scan to Download
Listen It

-
Links
: The simplest technique, where teams link to each other’s pages.
-
Application shell
: Enhances user experience by enabling client-side transitions without full page reloads.

9.1.2 Composition techniques

Micro frontends often involve integrating UI components from different teams. Two primary approaches include:
-
Server-side integration
: Assembles markup before sending it to the client.
-
Client-side integration
: Uses custom elements and JavaScript to dynamically render UI components in the browser.
-
Iframe and Ajax
are hybrid techniques combining aspects of both approaches.

More Free Book
Scan to Download
Listen It

9.1.3 High-level architectures

Different high-level architectures range from simple linked pages to complex unified SPAs. Key architectures are:

-
Linked pages
: Simple navigation via hyperlinks.

-
Server routing
: A shared server manages navigation between teams.

-
Linked SPAs
: Combines linked pages with client-side rendered content for faster navigation.

-
Unified SPA
: Integrates multiple SPAs into a single application shell for a cohesive user experience.

-
Unified Universal SPA
: Adds server-side rendering for speed while maintaining single-page application benefits.

9.2 Comparing complexity

More Free Book
Scan to Download
Listen It

Architectural choices drastically affect project complexity, influencing initial setup, maintenance, coupling, developer skill requirements, and debugging challenges. Simpler architectures are generally preferred unless specific complexities justify more advanced solutions.

9.2.1 Heterogeneous architectures

Different teams can adopt varied architectures based on their needs, leading to flexibility but potentially complicating integration and collaboration.

9.3 Are you building a site or an app?

The nature of the project influences architectural decisions, particularly regarding server-side versus client-side rendering. The Documents-to-Applications Continuum helps visualize this distinction and informs the choice of technology.

9.3.1 The Documents-to-Applications Continuum

Projects can be positioned on a spectrum ranging from

More Free Book
Scan to Download
Listen It

content-centric (e.g., blog) to function-centric (e.g., web applications). This positioning assists in prioritizing development approaches.

9.4 Picking the right architecture and integration technique

A decision tree should be used to identify the most suitable architecture and integration techniques based on project requirements. Key questions revolve around isolation, performance needs, instant feedback, and navigation type.

Summary

Having a shared vocabulary is crucial for common understanding among teams. The choice between architectures should be guided by the specific requirements of the project, the desired level of complexity, and development capabilities. It's acceptable to mix architectures as long as integration techniques are efficient and meet the project’s needs.

More Free Book
Scan to Download
Listen It

Example

Key Point:Shared Vocabulary Enhances Collaboration

Example:Imagine you're leading a team to develop a complex web application using micro frontends. Without a common vocabulary, when your designers talk about 'components' and developers refer to 'elements', confusion ensues, causing delays. Establishing a shared understanding ensures everyone's on the same page about integration techniques and project requirements, streamlining communication and boosting productivity. This is essential for making effective architectural decisions that align with your project's unique needs.

More Free Book

Scan to Download

Listen It

Chapter 12 Summary : Part 3. How to be fast, consistent, and effective

Part 3: How to be Fast, Consistent, and Effective

You've learned the integration techniques to build a micro frontends application. However, to make your project successful, there are topics beyond integration that you need to address. We touched on the aspects of performance, visual consistency, and team responsibility at various points in the preceding chapters. In this next part of the book, we will cast a brighter light on these complementary aspects.

Chapter Overviews

Chapter 10: Asset Loading

We start with a technical topic: asset loading. Loading the right script and style code for the different micro frontends can be surprisingly challenging—especially if you want to adhere to performance best practices without sacrificing team

More Free Book
Scan to Download
Listen It

autonomy.

Chapter 11: Performance

This chapter dives deeper into performance. You'll see common pitfalls and learn strategies to build and maintain a fast-loading site even when the UI comes from different teams.

Chapter 12: Consistency in Look and Feel

One of the biggest criticisms of micro frontends is ensuring a consistent look and feel for the end user. Design systems are essential in addressing this. However, establishing a shared design system that doesn't interfere with the autonomy goals of micro frontends can be challenging.

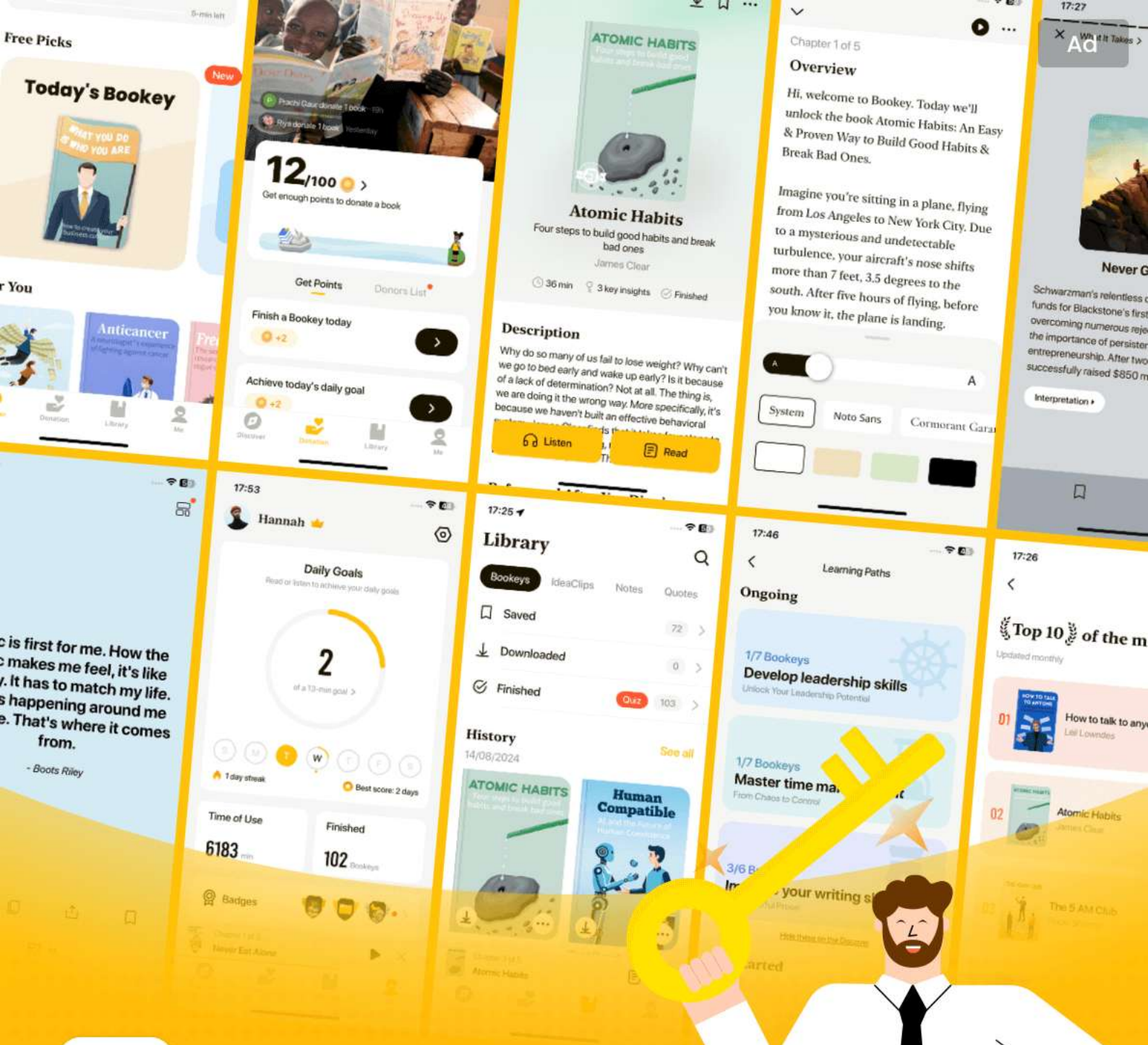
Chapter 13: Organizational Implications

Install Bookey App to Unlock Full Text and Audio

More Free Book

Scan to Download

Listen It



World' best ideas unlock your potential

Free Trial with Bookey

Scan to download

Download on the App Store

GET IT ON Google Play

Chapter 13 Summary : 10 Asset loading

Asset Loading

This chapter covers various strategies and considerations for loading assets in a micro frontends context. Key topics include:
1. Solving common asset loading challenges.
2. Techniques for handling cacheability and synchronization from different teams.
3. Choosing an appropriate bundling strategy (many smaller bundles vs. fewer large ones).
4. Utilizing on-demand loading effectively in micro frontends.
While previous chapters focused on integration techniques, this chapter delves into how to load the associated assets of micro frontends, addressing several important questions:
- How can teams independently deploy their micro frontend and the necessary assets?
- What strategies can be implemented for cache busting to enhance cacheability without tight coupling?
- How do you ensure that loaded CSS and JavaScript align with the server-generated markup?

More Free Book
Scan to Download
Listen It

- What is the ideal granularity for your bundles?

Asset Referencing Strategies

The chapter introduces methods to integrate assets into pages, primarily using traditional `<link>` and `<script>` tags. Despite the existence of module loaders, modern browsers support ES Modules, allowing for a simpler approach without additional libraries. For this context, we assume each team generates one JavaScript and one CSS file for its micro frontend.

Direct Referencing

Direct referencing involves adding references for micro frontends from other teams. This is akin to importing files in languages like Java or JavaScript. Using an app shell model requires a single HTML document to manage asset loading for all micro frontends.

1. **Preloading vs. On-Demand Loading**:
   - A straightforward method is to preload all assets from all teams.

More Free Book
Scan to Download
Listen It

- A more efficient approach is to load assets on-demand, as needed by the user, exemplified by the meta-framework single-spa’s implementation.

Revisiting assets in "The Tractor Store" example, Team Decide has included direct references to the necessary assets in their documentation. For optimal rendering performance, stylesheets should be included in the `<head>`, while script tags should be loaded asynchronously at the end of the `<body>`, as illustrated by Team Decide’s product page markup.

More Free Book
Scan to Download
Listen It

Chapter 14 Summary : 11 Performance is key

Performance in Micro Frontends

Key Concepts

This chapter focuses on optimizing performance in micro frontends, covering:
- Measuring performance effects when multiple micro frontends are present on a single page.
- Identifying regressions and bottlenecks, and assigning responsibility to the respective teams.
- Recognizing common performance issues tied to micro frontend architecture.
- Strategies for reducing JavaScript load by sharing larger vendor libraries among teams.

Architecting for Performance

Finn from Tractor Models, Inc. initiated a performance-first

More Free Book
Scan to Download
Listen It

approach by defining guidelines during team meetings. This included limiting page weight to 1MB and ensuring the viewport renders in one second under optimal conditions and three seconds on 3G networks.
Different metrics for performance were established, depending on the user context, with overarching rules set to provide a baseline. Teams could pick metrics specific to their subdomains.

Performance Budgets

Defining performance budgets helps establish a culture focused on performance. A concrete metric limit (e.g., total page size) encourages ongoing measurement with tools like Lighthouse and speed monitoring systems. If a new feature violates the budget, discussions among the team help strategize optimizations or rollbacks.
Performance can be evaluated from both individual micro frontend contributions and page-level responsibilities. This fosters accountability and a responsive structure among teams.

Attributing Slowdowns

More Free Book
Scan to Download
Listen It

Effective monitoring tools and practices are crucial for identifying performance slowdowns in micro frontends. For example, monitoring average load times, response times, and tracking changes with visual tools aids teams in diagnosing issues promptly.
Communication between teams is necessary to resolve any issues after identifying the source of slowdowns. The observability tools and metrics should provide central visibility into performance impacts from various components.

Performance Benefits

While there are challenges, the micro frontends approach leads to several advantages as well:
1.
Built-in Code Splitting:
Smaller JavaScript bundles enhance caching and reduce the processing load on browsers.
2.
Use Case Optimization:
Teams working on specific features can tailor optimizations without broader organizational constraints.
3.

More Free Book
Scan to Download
Listen It

Easier Changes:
Isolated teams can clean up codebases without risks of unintended consequences across a monolithic structure.

Vendor Libraries and Redundancy

Addressing redundancy is crucial since duplicating large libraries across micro frontends can lead to inefficiencies. Teams at Tractor Models, Inc. agreed on a shared framework to optimize this redundancy.

1. **Cost of Autonomy:**
Teams achieve efficiency through shared frameworks while maintaining some autonomy to adapt as necessary.
2. **Choosing Smaller Libraries:**
Opting for lightweight frameworks reduces overall download size and improves performance.
3. **Global Versioning:**
Utilizing a global script for shared frameworks can simplify dependency management and improve load efficiency.

By implementing these strategies, performance in micro frontends can be significantly enhanced without sacrificing

More Free Book
Scan to Download
Listen It

team autonomy or development speed. Evaluation of individual performance impacts paired with collaborative ownership creates a healthier environment for continuous improvement.

More Free Book
Scan to Download
Listen It

Chapter 15 Summary : 12 User interface and design system

User Interface and Design System

This chapter covers:
- How a design system can help create a consistent user experience.
- The development of a design system and its impact on micro frontend teams' autonomy.
- Technical challenges in building a technology-agnostic pattern library.
- Deciding whether a component should be centralized in a pattern library or remain with a product team.

In a micro frontend architecture, each team is responsible for its part of the frontend, leading to challenges in achieving a consistent look and feel. A design system can provide shared design elements such as color palettes and typography, ensuring coherence across various teams. While design discussions are often undervalued, a structured approach is essential for success in distributed architectures. A well-planned design system enables teams to build similar

More Free Book
Scan to Download
Listen It

user interfaces, though it introduces a degree of coupling as all teams need to remain compatible with it.

Why a Design System?

A design system is an organized approach to design in the context of increasing complexity in web applications. It includes elements like design tokens, interface components, patterns, and usage guidelines. It can unify teams under a shared language, improve development speed, and scale effectively as more teams utilize it. A robust design system often requires significant upfront investment but pays off in terms of consistency and efficiency in the long run.

Central Design System vs. Autonomous Teams

-

Creation of a Design System:

Install Bookey App to Unlock Full Text and Audio

More Free Book

Scan to Download

Listen It

Ad

Scan to Download

Try Bookey App to read 1000+ summary of world best books

Unlock 1000+ Titles, 80+ Topics

New titles added every week

Brand | Leadership & Collaboration | Time Management | Relationship & Communication

ness Strategy | Creativity | Public | Money & Investing | Know Yourself | Positive P

Entrepreneurship | World History | Parent-Child Communication | Self-care | Mind & Spi

Insights of world best books

Free Trial with Bookey

Chapter 16 Summary : 13 Teams and boundaries

1. Teams and Boundaries

This chapter focuses on structuring teams to maximize the benefits of micro frontends architecture, emphasizing the importance of knowledge-sharing, addressing crosscutting concerns, and the impact of diverse technology landscapes on team dynamics.

1.1 Aligning Systems and Teams

Conway's Law suggests that an organization’s communication structure reflects in its technical systems. Effective micro frontend architecture aligns team boundaries with vertical applications, ensuring better modular solutions.

1.1.1 Identifying Team Boundaries

Several methods for identifying beneficial team structures include:

More Free Book
Scan to Download
Listen It

-
Domain-driven design (DDD)
: Identifies bounded contexts that can form their own micro frontend teams.
-
User-centered design
: Structures teams based on user needs throughout the buying process.
-
Examining existing page structures
: Analyzing current projects' page types can offer insights into suitable team groupings.

1.1.2 Team Depth

Micro frontend teams should ideally encompass the entire stack (frontend, backend, operations) to enhance feature delivery and decision-making autonomy. Benefits of full-stack teams include creativity, reduced coordination needs, and the ability to adapt quickly to market changes.

1.1.3 Cultural Change

Transitioning to a vertical architecture fosters a user-focused

More Free Book
Scan to Download
Listen It

culture. Team ownership and empowerment lead to faster decision-making and better product value. A culture based on agile values is crucial for success.

1.2 Sharing Knowledge

The cross-functional team structure can lead to challenges like reinventing solutions across teams. Effective strategies include:

- **Community of practice (CoP)**
: Groups where individuals in similar roles can share knowledge and support.
- **Learning and enabling**
: Teams should identify strengths and weaknesses, encouraging skill development and knowledge sharing.
- **Present your work**
: Regular presentations of work done by different teams foster understanding and collaboration.

1.3 Cross-Cutting Concerns

More Free Book
Scan to Download
Listen It

Some concerns necessitate shared infrastructure to avoid redundancy and inefficiency, such as:

-

Central Infrastructure

: Shared tools should be adopted to streamline processes.

-

Specialized Component Team

: Teams dedicated to developing specific needed services can help avoid common issues among product teams.

-

Global Agreements and Conventions

: Documented agreements on practices can unify efforts without shared services.

1.4 Technology Diversity

Micro frontends allow for diverse technology stacks, which can be beneficial yet burdensome. Strategies to manage this include:

-

Toolbox and defaults

: Providing vetted options can guide teams' technology choices while allowing flexibility.

-

More Free Book
Scan to Download
Listen It

Frontend blueprint
: A shared starting point for new teams expedites the integration process and ensures adherence to high-level architecture.
-
Encourage copying
: Sharing smaller code fragments can maintain autonomy while reducing redundancy in simple tasks.
-
Value of similarity
: Utilizing similar technologies across teams fosters collaboration and eases transitions between team members.

Summary

Establishing a successful micro frontends architecture relies on aligning team structures with software systems, intelligently determining team boundaries, and fostering a culture of ownership and user-focus. Knowledge sharing through communities of practice and well-defined infrastructure can facilitate collaboration and reduce redundancy. Embracing diverse technology options while maintaining a sensible framework aids in effective development.

More Free Book
Scan to Download
Listen It

Chapter 17 Summary : 14 Migration, local development, and testing

Migration, Local Development, and Testing

This chapter discusses migrating a monolithic application to a micro frontends architecture, setting up a local development environment, and implementing automated testing.

14.1 Migration

Migrating a non-trivial project to micro frontends can be daunting. The chapter presents three migration strategies and highlights the importance of understanding the complexity and effort involved in such transitions.

14.1.1 Proof of Concept and Building a Lighthouse

Building a single feature as its own micro frontends application can serve as proof of concept. This example demonstrates how to integrate new teams and features into an

More Free Book
Scan to Download
Listen It

existing monolith, using a practical example from an e-commerce company, Miniature Farming Industries.

14.1.2 Strategy #1: Slice-by-Slice

The slice-by-slice approach gradually replaces monolithic functionalities with micro frontends, allowing for incremental migration and reduced risk. Teams establish clear boundaries, migrate features step by step using the Strangler Fig Pattern, ensuring that the application remains functional throughout the process.

14.1.3 Strategy #2: Frontend First

This approach prioritizes frontend migration over backend changes, simplifying development by avoiding the mixing of old and new frontend code. This method progresses through two phases: first rebuilding the frontend and then backend API migrations.

14.1.4 Strategy #3: Greenfield and Big Bang

The greenfield approach allows the new system to be built independently of the monolith and switched over all at once.

More Free Book
Scan to Download
Listen It

While this option is conceptually appealing, it carries significant risk due to the lack of real-time user feedback during development.

14.2 Local Development

Moving into day-to-day development, the chapter explains the challenges of local development in a micro frontends environment and why teams should avoid running code from other teams.

14.2.1 Don’t Run Another Team’s Code

To maintain independence, each team should focus solely on their codebase, avoiding complexities that arise from needing to manage dependencies or updates from other teams.

14.2.2 Mocking Fragments

Teams can use mock versions of other teams' fragments to facilitate local development. This allows for smoother integration and debugging without relying on shared code.

14.2.3 Fragments in Isolation

More Free Book
Scan to Download
Listen It

Each team can create sandbox pages for their fragments to develop and test features in isolation, allowing them to simulate user interactions and ensure proper functionality.

14.2.4 Pulling Other Teams' Micro Frontends from Staging or Production

In cases where mocking isn't sufficient, teams can reference the staging or production versions of other teams' micro frontends to ensure compatibility and debug issues.

14.3 Testing

Automated testing is crucial in a micro frontends architecture. Teams conduct unit tests, service tests, and end-to-end tests, focusing on isolating their components as much as possible, while maintaining an awareness of cross-team interactions.

Testing Strategy Overview

Isolated testing is preferred, but critical integration points should undergo full integration testing. Each team is

More Free Book
Scan to Download
Listen It

responsible for testing their components and can test integrations with neighboring teams to ensure functionality without introducing unnecessary complexity.

Summary

Micro frontends facilitate gradual migrations while allowing teams to operate independently. Efforts should focus on maintaining isolation during local development and testing, leveraging mocks and sandbox environments, and ensuring collaboration through well-defined testing protocols.

More Free Book
Scan to Download
Listen It

Critical Thinking

Key Point:The chapter emphasizes the need for careful consideration in migrating to a micro frontends architecture.

Critical Interpretation:While the author posits that micro frontends can simplify migration and enhance team independence, skeptics may argue that the complexity introduced in managing multiple microservices could outweigh the purported benefits. There are numerous examples in industry literature, such as Martin Fowler's writings on microservices, that highlight the potential pitfalls of distributed systems, urging teams to weigh scalability against the added overhead of integration and maintenance challenges.

More Free Book
Scan to Download
Listen It

Ad

Scan to Download

Why Bookey is must have App for Book Lovers

30min Content
The deeper and clearer interpretation we provide, the better grasp of each title you have.

Text and Audio format
Absorb knowledge even in fragmented time.

Quiz
Check whether you have mastered what you just learned.

And more
Multiple Voices & fonts, Mind Map, Quotes, IdeaClips...

Free Trial with Bookey

Best Quotes from Micro Frontends in Action by Michael Geers with Page Numbers

View on Bookey Website and Generate Beautiful Quote Images

Chapter 1 | Quotes From Pages 1-43

1. In 2016, having the task of rebuilding an e-commerce system for a department store chain... the major pain-point was an organizational one: adding new features took a long time and often broke unrelated parts of the system.
2. We opted for an architecture we called verticalization: the establishment of different cross-functional teams that build and evolve a specific area of the shop from database to user interface.
3. The introduction of this name (micro frontends) made it possible for the development community to share best practices, techniques, and tools around this architecture.
4. I had long consultations with friends and family and some sleepless nights, but in the end, I accepted the challenge.

More Free Book
Scan to Download
Listen It

After all, I like explaining things.

5. It takes a village to create a book like this.

Chapter 2 | Quotes From Pages 47-117

1. At some point, adding new developers to a team does not increase productivity.
2. Each slice is built from the database to the user interface and run by a dedicated team.
3. The micro frontends model optimizes for implementing features by moving all necessary people closer together.
4. Cross-functional teams come with the added benefit that all members are directly involved in feature development.
5. A micro frontend can be a complete page or a fragment that other teams include.
6. Every team should have a descriptive name and a clear user-focused mission.
7. The reasoning behind this shared-nothing architecture is that the costs associated with these redundancies are smaller than the negative impacts that inter-team dependencies introduce.

More Free Book
Scan to Download
Listen It

8. When you are building an application of a specific size and want to stay competitive, it’s essential to be able to move to new technologies when they provide value for your team.

Chapter 3 | Quotes From Pages 118-171

1. Being able to work on a complex application with multiple teams in parallel is the essential feature of micro frontends.
2. No shared infrastructure, libraries, or code conventions are required.
3. There is no 'gold standard' or 'best integration technique.' It’s all about making the right trade-offs for your use case.
4. The teams can choose the technology that’s best for the job.
5. The URL is our contract between the teams.

More Free Book
Scan to Download
Listen It

Scan to Download

Download Bookey App to enjoy

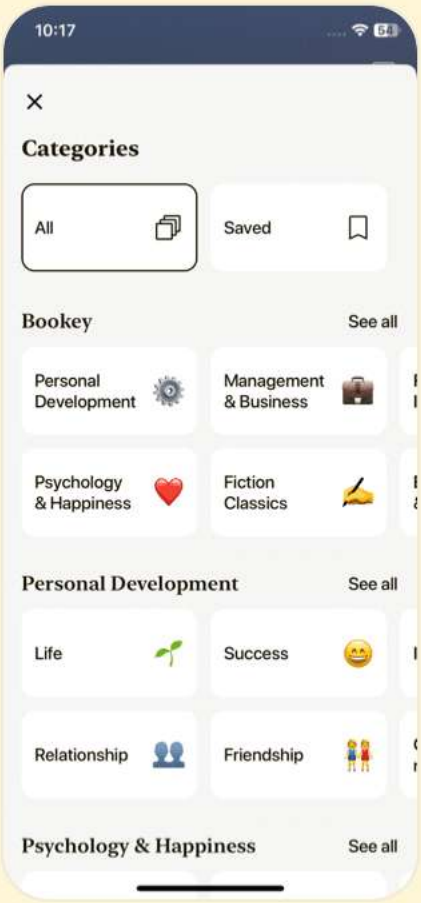
1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Download on the App Store

GET IT ON Google Play



Chapter 4 | Quotes From Pages 172-174

1. Adoption of the micro frontends approach is spreading in the software industry.
2. Throughout the following chapters, we'll focus on existing web standards and leverage native browser features wherever possible.
3. Understanding the core concepts is vital for building a successful project, even if you decide to pick up a micro frontends library later.
4. I've arranged the chapters by complexity. We start with simple techniques and work our way up to more sophisticated ones.

Chapter 5 | Quotes From Pages 175-242

1. We'll also configure a shared web server to expose all applications through a single domain.
2. They decide to address this issue in the upcoming iteration.
3. All data and styles are already there from the iframe integration.
4. Shipping a link tag together with the actual content is not

More Free Book
Scan to Download
Listen It

always optimal.

Chapter 6 | Quotes From Pages 243-323

1. The most significant benefit of server-side integration is that the page is already fully assembled when it reaches the customer’s browser.
2. When an upstream becomes slow or doesn’t respond at all, Nginx stops waiting and delivers the site without the includes.
3. Server-side integration optimizes the critical paths.
4. Thinking about fallbacks and timeouts is crucial when you implement server-side composition.
5. The critical difference is that latency inside one data center is magnitudes smaller and more predictable.

More Free Book
Scan to Download
Listen It

Scan to Download

Download Bookey App to enjoy

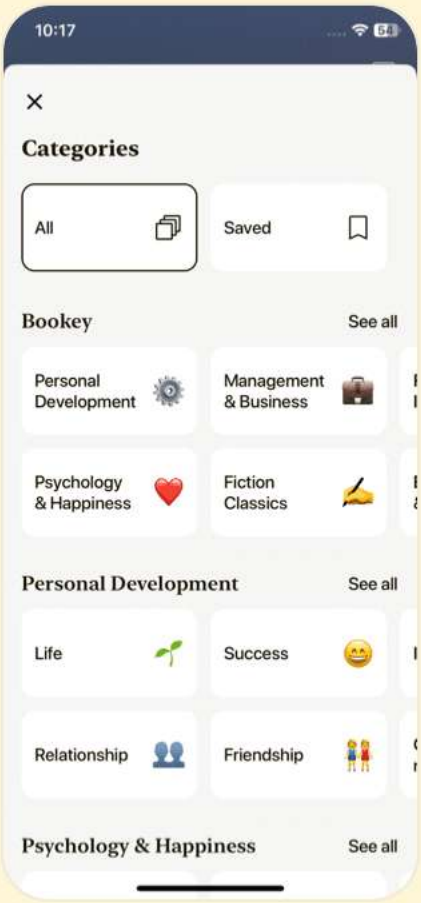
1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Download on the App Store

GET IT ON Google Play



Chapter 7 | Quotes From Pages 324-365

1. People spend more time on sites that react quickly and feel app-like.
2. In a micro frontends architecture, we want the user interfaces from the different teams to be self-contained and independently upgradable.
3. The Web Components spec introduces a neutral and standardized component model.
4. Styles that are defined in the Shadow DOM stay in the Shadow DOM. Nothing leaks out to affect other parts of the page.
5. Web Components are a widely implemented web standard, providing extra isolation features that make your micro frontends more robust.
6. If you are building an interactive, app-like application where user interfaces from different teams must be integrated on one screen, Web Components are a reliable basis.

Chapter 8 | Quotes From Pages 366-417

More Free Book
Scan to Download
Listen It

1. If you've chosen good team boundaries, you'll learn more about how to do it in chapter 13. There should be little need for extensive cross-UI communication in the browser.
2. To accomplish a task, a customer is ideally only in contact with the user interface from one team.
3. Using the browser's event mechanism has multiple benefits: Custom Events can have high-level names that reflect your domain language.
4. Child-parent communication can be implemented by using the browser's built-in event mechanism.
5. The introduction of the platinum editions resulted in a lot of controversial discussions in the Tractor Model, Inc., user forum.
6. For a clean solution, the animation has to be developed by Team Decide. To accomplish this, both teams have to work together through a clearly defined contract.
7. With this model, you introduce a global communication channel. Fragments can publish events to the channel.

More Free Book
Scan to Download
Listen It

Other fragments can subscribe to these events and react to them.

Chapter 9 | Quotes From Pages 418-492

1. The application shell acts as a parent application for all micro frontends.
2. It should not contain any business logic.
3. The fact that a team uses a SPA for its pages is an implementation detail.
4. All incoming requests arrive there. It selects the micro frontend the user wants to see and renders it in the <body> of the document.
5. Moving to one shared framework would do the opposite.

More Free Book
Scan to Download
Listen It

Scan to Download

Download Bookey App to enjoy

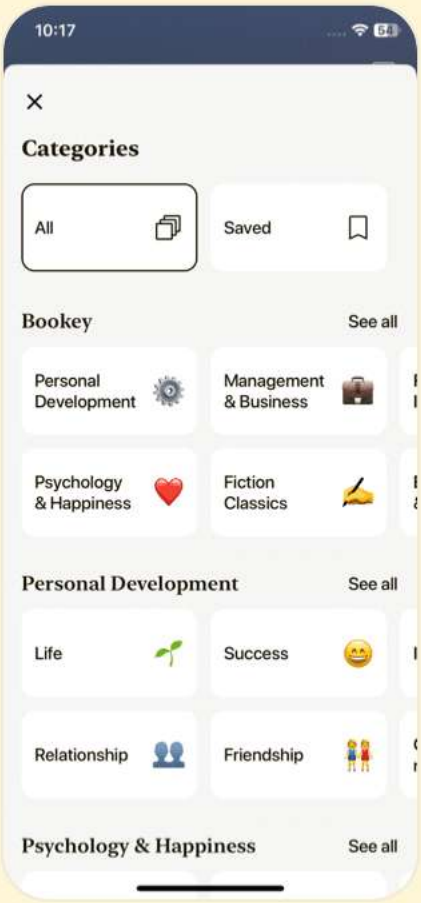
1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Download on the App Store

GET IT ON Google Play



Chapter 10 | Quotes From Pages 493-524

1. Universal composition is the combination of a server- and a client-side composition technique.
2. Universal rendering combines the benefits of server and client rendering: fast first-page load and quick response to user input.
3. Each team must be able to render its micro-frontend via an HTTP endpoint on the server and also make it available via JavaScript in the browser.
4. Building an application with progressive enhancement principles in mind requires a little more thinking and testing than relying on the fact that JavaScript always works.

Chapter 11 | Quotes From Pages 525-576

1. Establishing a shared vocabulary across all teams avoids misunderstandings.
2. The Documents-to-Applications Continuum is a good mental model for identifying whether your project is more content- or behavior-centric.

More Free Book
Scan to Download
Listen It

3. Not all teams need to adopt the same architecture. Some parts of your application might be document-centric; others more behavior-centric.
4. Try to pick the simplest architecture that is reasonable for your business.

Chapter 12 | Quotes From Pages 577-579

1. Loading the right script and style code for the different micro frontends can be surprisingly challenging--especially if you want to adhere to performance best practices without sacrificing team autonomy.
2. Design systems are essential in addressing this issue.
3. Establishing a shared design system so that it doesn't interfere with the autonomy goals of micro frontends is not without challenges.
4. You'll learn how to find sound team boundaries, organize inter-team knowledge transfer, and deal with shared infrastructure.
5. The last chapter covers migrating to micro frontends, tips

More Free Book
Scan to Download
Listen It

for local development, and some patterns for effective testing.

More Free Book
Scan to Download
Listen It

Scan to Download

Download Bookey App to enjoy

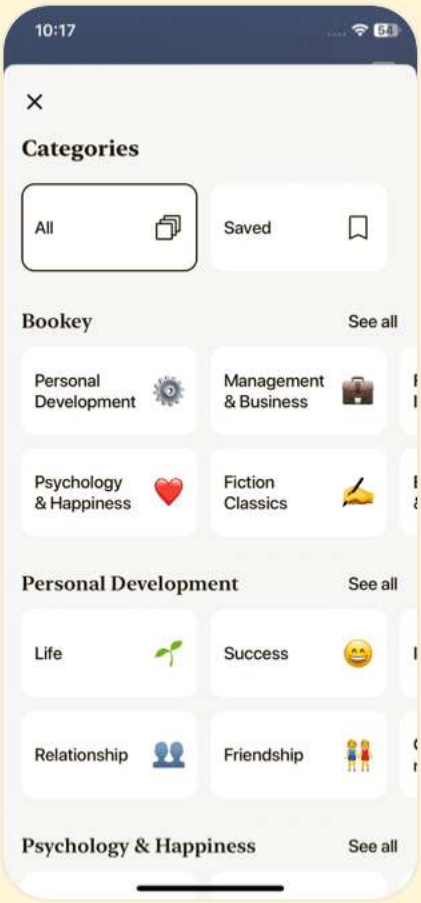
1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Download on the App Store

GET IT ON Google Play



Chapter 13 | Quotes From Pages 580-642

1. How can we ensure that teams can deploy a micro frontend and the needed assets on their own?
2. How do you implement cache busting to improve cacheability without introducing tight coupling?
3. How do you ensure that the loaded CSS and JS always fits the server-generated markup?
4. How coarse or fine-grained should your bundles be?
5. How can on-demand loading techniques help in reducing the upfront asset data the browser needs to process?

Chapter 14 | Quotes From Pages 643-704

1. Having worked in micro frontend projects over the last years, my initial worries faded quickly.
2. Architecting for excellent performance was a top priority from the start, and not an afterthought.
3. A performance budget is a perfect tool to establish a performance-oriented culture inside a team.
4. This focus on teasers allows them to move quicker. No compromises.

More Free Book
Scan to Download
Listen It

5. The responsibility is very clear. When the product page is slow, it’s Team Decide that needs to become active.
6. Investing in proper monitoring will make finding issues much more manageable.
7. Delivering the code into smaller chunks has benefits: Cacheability, fewer long-running tasks, and on-demand loading.
8. The way you think about and develop features in a micro frontends project naturally guides you toward this structure.
9. Autonomy inherently comes with the cost of accepting redundancy.
10. Having clearly isolated micro frontends reduces the risk of cleaning up dramatically.

Chapter 15 | Quotes From Pages 705-779

1. In a distributed architecture like this, it’s essential to have a proper plan for managing your design from the start.
2. A design system enables them to build interfaces that have a similar look. However, a design system also introduces

More Free Book
Scan to Download
Listen It

coupling because every team has to be compatible with it.

3. The goal of a design system team should never be to create the most beautiful, best-documented, or most versatile design system on the market. The objective of a design system team should be supporting the product teams as best as possible.
4. It’s important to set appropriate expectations in management. The bulk of the design system work will be in the first few months, but the work doesn’t stop then.
5. Don't let your design system join the zombie army.

More Free Book
Scan to Download
Listen It

Scan to Download

Download Bookey App to enjoy

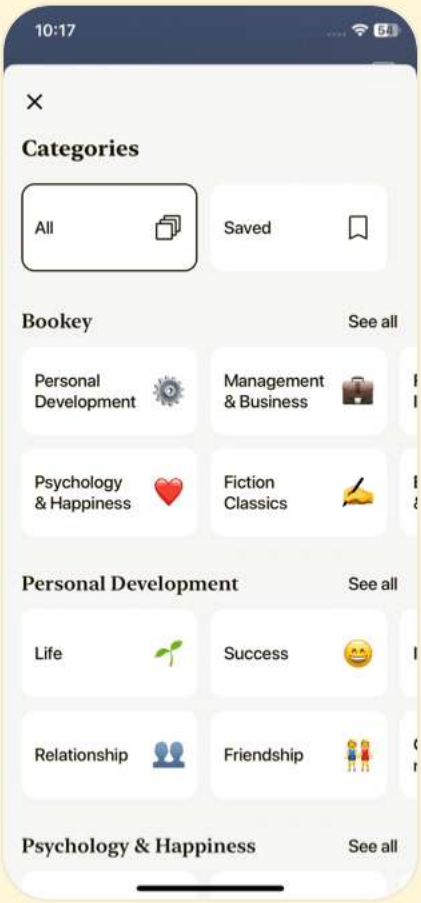
1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Download on the App Store

GET IT ON Google Play



Chapter 16 | Quotes From Pages 780-838

1. It’s not about the software. It’s about the people designing and building it.
2. Make sure the organization is compatible with the product architecture.
3. The vertical architecture plays well with having a user-focused culture.
4. Only do it if you are willing to run it as a successful (internal) open-source project.
5. Establishing a shared set of infrastructure that all teams can use is a good idea.

Chapter 17 | Quotes From Pages 839-884

1. Micro frontends is not the first architecture for most companies. It’s something you migrate to because the old architecture has trouble keeping up with new demands like increasing team size or high demand for features.
2. Having a somewhat realistic idea of the complexity and effort a migration takes is vital to set expectations and

More Free Book
Scan to Download
Listen It

calculate costs.

3. The 'proof of concept' can become the role model other teams can follow to build new features.
4. The main benefit of this incremental migration approach is that it introduces little risk. The newly created software goes into production regularly.
5. Investing some effort in good mocks will make your life easier and can save a lot of time down the road.
6. Every team tests its application on different levels. They'll have a bunch of fast-running unit and service tests and a couple of browser-based end-to-end tests.

More Free Book
Scan to Download
Listen It

Scan to Download

Download Bookey App to enjoy

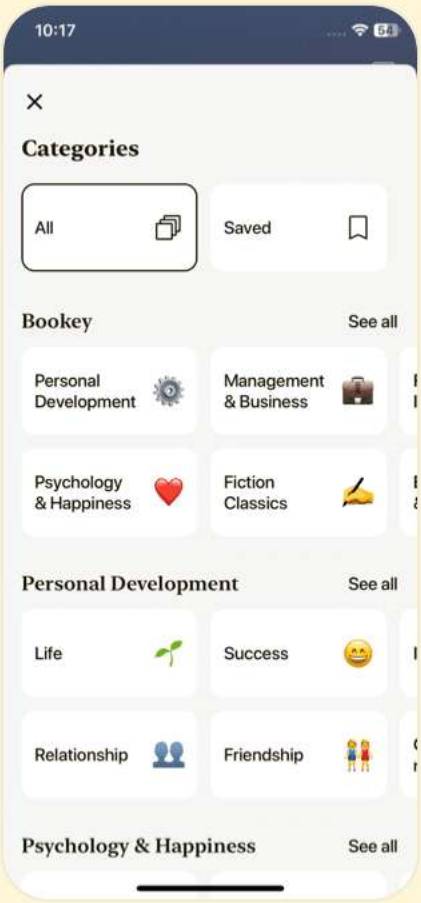
1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Download on the App Store

GET IT ON Google Play



Micro Frontends in Action Questions

View on Bookey Website

Chapter 1 | Micro Frontends in Action| Q&A

1.Question

What challenges prompted the development of micro frontends?

Answer:The need to improve the performance and speed of feature development in a monolithic architecture where adding new features would often break unrelated parts of the system, especially in large organizations. This was underscored by the experience of teams struggling with coordination as they scaled up for digitization.

2.Question

Why is the term 'micro frontends' significant?

Answer:The term 'micro frontends' was introduced in 2016, which allowed the development community to share best practices, techniques, and tools specifically related to this architecture, facilitating broader discussion and improvement

More Free Book
Scan to Download
Listen It

in the methodology.

3.Question

How does the book aim to support different technical backgrounds?

Answer:Although the book focuses on frontend development, it is designed for anyone with a basic understanding of HTML, CSS, JavaScript, and networking, ensuring that backend developers and software architects can also find value in its insights.

4.Question

What is the overall structure of the book?

Answer:The book is divided into three parts, covering the fundamentals of micro frontends, frontend integration techniques, and best practices for performance and consistent user interfaces. It guides readers through practical applications with progressively complex examples.

5.Question

What key lesson did the author learn from his experiences?

Answer:The author learned the importance of establishing

More Free Book
Scan to Download
Listen It

independent, cross-functional teams that can work autonomously on different parts of a system without interfering with one another, thereby enhancing parallel development and overall productivity.

6.Question

What personal challenges did the author face when writing this book?

Answer:The author initially hesitated to write the book due to a lack of experience and a preference for practical work over reading and writing. Ultimately, he embraced the challenge as an opportunity for personal growth, demonstrating that stepping out of one's comfort zone can yield positive outcomes.

7.Question

How does the book address the integration of user interfaces from different teams?

Answer:The book provides detailed approaches for routing and composition techniques that help different teams' user interfaces come together in the browser, emphasizing

More Free Book
Scan to Download
Listen It

practical methods to achieve this integration smoothly.

8.Question

What is verticalization in the context of micro frontends?

Answer:Verticalization refers to the architectural approach adopted by the author and his team, wherein cross-functional teams are set up to progress on specific areas of a project independently, allowing for specialized development without conflicts.

9.Question

What role did community feedback play in the author's journey?

Answer:Community feedback not only motivated the author, as evidenced by the engagement from developers worldwide, but also significantly shaped the content of the book and its practical applications, illustrating the value of collaboration in software development.

10.Question

What can readers expect to learn by the end of the book?

Answer:Readers can expect to grasp fundamental mechanisms of integrating micro frontends, achieving good

More Free Book
Scan to Download
Listen It

performance and a consistent design across applications, and understanding how to effectively organize teams and processes in a distributed architecture.

Chapter 2 | 1 What are micro frontends?| Q&A

1.Question

What are micro frontends and why are they important?

Answer:Micro frontends are an architectural approach that breaks down a web application into smaller, independent parts, allowing teams to work autonomously on different features from the database to the user interface. This approach is essential as it helps in scaling frontend development, reduces complexity, promotes team ownership, and fosters faster feature development.

2.Question

How do micro frontends compare to traditional architectures?

Answer:Unlike traditional architectures, which often involve horizontal layering (separating frontend and backend teams),

More Free Book
Scan to Download
Listen It

micro frontends encourage vertical slicing where each team is responsible for an end-to-end feature. This reduces inter-team dependencies and speeds up the development process.

3.Question

What are the main benefits of adopting micro frontends?

Answer:The main benefits include: 1) Faster feature development because teams can work independently; 2) Easier frontend upgrades as teams manage their complete technology stack; 3) Increased customer focus, with teams directly shipping features to users.

4.Question

What challenges might arise from implementing micro frontends?

Answer:Challenges include potential redundancy, as multiple teams may duplicate efforts in setting up infrastructure; consistency issues between different technology stacks; and the risk of increased code size affecting performance.

5.Question

When is it appropriate to adopt a micro frontends

More Free Book
Scan to Download
Listen It

architecture?

Answer:Micro frontends are particularly suitable for medium to large projects where multiple teams are involved, as they help in managing complexity and scaling up development efforts. They excel in web applications but may not be as beneficial in smaller projects where communication is not an issue.

6.Question

How do micro frontends enhance team autonomy?

Answer:Micro frontends empower teams to choose their technology stacks and manage their deployment processes independently, enabling faster iterations and adaptations to new technologies without lengthy coordination with other teams.

7.Question

What are some best practices for implementing micro frontends?

Answer:Best practices include establishing clear team missions aligned with customer needs, maintaining a shared

More Free Book
Scan to Download
Listen It

design system for consistency, and focusing on web performance from the outset to manage the increased complexity that micro frontends can introduce.

8.Question

What are 'fragments' in the context of micro frontends?

Answer:Fragments are self-contained mini-applications or components that can be embedded within larger pages. They allow teams to share specific functionality without needing to replicate entire pages, thereby enhancing modularity and reducing redundancy.

Chapter 3 | 2 My first micro frontends project| Q&A

1.Question

What is the primary benefit of using micro frontends in application development?

Answer:The primary benefit of using micro frontends is the ability for multiple teams to work independently and in parallel on separate aspects of a complex application. This reduces internal dependencies, allowing teams to focus on their

More Free Book
Scan to Download
Listen It

specific goals and technologies, which enhances flexibility and speed of development.

2.Question

How does Team Decide and Team Inspire interact in the Tractor Store project?

Answer:Team Decide is responsible for creating the product detail pages that display tractor models, while Team Inspire handles the recommendation system. They interact by linking their respective pages, allowing users to navigate seamlessly from a product page to a recommendations page.

3.Question

What technology choices did Team Decide and Team Inspire make, and why?

Answer:Team Decide opted for a MongoDB database and Node.js for a server-rendered application, which suits their need for handling product data. Team Inspire chose a Python-based stack for implementing machine learning techniques to provide personalized recommendations. Their technology choices reflect their specific project requirements.

More Free Book
Scan to Download
Listen It

4.Question

What does minimal coupling between teams mean in the context of micro frontends?

Answer:Minimal coupling means that teams can develop their UIs without relying on shared infrastructure or code from other teams. Each team is free to choose their own technology stack and deployment process, which mitigates risks and allows for greater autonomy.

5.Question

In what ways can technology diversity be beneficial in a micro frontends architecture?

Answer:Technology diversity allows teams to select the best tools for their specific tasks, leading to optimized solutions. It enables teams to innovate and experiment with new technologies suited for their features, enhancing overall application quality and performance.

6.Question

What role does the 'contract' between teams play in micro frontends?

Answer:The 'contract' is established through URL patterns

More Free Book
Scan to Download
Listen It

that teams publish for their pages. This ensures that each team knows how to link their features to others correctly without needing to understand each other's internal workings.

7.Question

Why is there no 'gold standard' for integration techniques in micro frontends?

Answer:There is no 'gold standard' because the choice of integration technique largely depends on the specific use case and trade-offs like complexity, performance, and team structure. Each scenario may require a different approach, and the best solution is context-dependent.

8.Question

How do independent deployment pipelines benefit teams in a micro frontends setup?

Answer:Independent deployment pipelines allow each team to deploy changes without affecting the others. This isolation means that updates in one team's application won't cause failures in another team's application, enhancing stability and facilitating more agile development cycles.

More Free Book
Scan to Download
Listen It

9.Question

What steps are involved in starting the Tractor Store project example code?

Answer:To start the example code, you must install the necessary Node.js dependencies, navigate to the root directory, and run the command `npm run [name_of_example]`. This will launch the application on specified ports and open it in your browser for interaction.

10.Question

What is the purpose of using an ad hoc HTTP server in the examples?

Answer:The ad hoc HTTP server is used to serve static files from each team's directory during development. It simplifies local testing of the applications by allowing teams to view their output directly in a web browser.

More Free Book
Scan to Download
Listen It

Ad

Scan to Download

App Store
Editors' Choice

22k 5 star review

Positive feedback

Sara Scholz

tes after each book summary
erstanding but also make the
and engaging. Bookey has
ding for me.

Fantastic!!!

Masood El Toure

I'm amazed by the variety of books and languages Bookey supports. It's not just an app, it's a gateway to global knowledge. Plus, earning points for charity is a big plus!

Fi
Ab
bo
to
my

José Botín

ding habit
p's design
ual growth

Love it!

Wonnie Tappkx

Bookey offers me time to go through the important parts of a book. It also gives me enough idea whether or not I should purchase the whole book version or not! It is easy to use!

Time saver!

Bookey is my go-to app for
summaries are concise, ins
curated. It's like having acc
right at my fingertips!

Awesome app!

Rahul Malviya

I love audiobooks but don't always have time to listen to the entire book! bookey allows me to get a summary of the highlights of the book I'm interested in!!! What a great concept !!!highly recommended!

Beautiful App

Alex Walk

This app is a lifesaver for book lovers with busy schedules. The summaries are spot on, and the mind maps help reinforce wh I've learned. Highly recommend!

Free Trial with Bookey

Chapter 4 | Part 2. Routing, composition, and communication| Q&A

1.Question

What are some key techniques to focus on when adopting micro frontends?

Answer:The key techniques to focus on include routing, composition, and communication. These elements are essential for both server-side and client-side rendered web applications and will help create a more sophisticated project.

2.Question

Why is it important to stick to web standards and native browser features?

Answer:Sticking to web standards and native browser features ensures stability and leverages the fundamental principles of web development. This approach has been proven valuable in various projects, making it easier to understand the core concepts underlying micro frontends.

3.Question

How does the author arrange the chapters in the book,

More Free Book
Scan to Download
Listen It

and why?

Answer:The chapters are arranged by complexity, starting with simple techniques and gradually introducing more sophisticated methods. This structured approach allows readers to build their understanding progressively, which is crucial for mastering the concepts related to micro frontends.

4.Question

What role do meta-frameworks and helper libraries play in micro frontend development?

Answer:Meta-frameworks and helper libraries help address common challenges in micro frontend development by offering additional abstractions and improving the developer experience. However, the author emphasizes understanding the underlying principles before diving into these tools.

5.Question

What is the ultimate goal of learning from this chapter about micro frontends?

Answer:The ultimate goal is to equip developers with the knowledge and techniques necessary to make informed

More Free Book
Scan to Download
Listen It

architectural decisions for their next project, ensuring successful and efficient micro frontend implementations.

Chapter 5 | 3 Composition with Ajax and server-side routing| Q&A

1.Question

What are the key benefits of integrating user interfaces via Ajax instead of iframes?

Answer:Integrating via Ajax provides stronger usability and performance. Unlike iframes, it allows for better semantic structure, which can positively impact search engine optimization (SEO).

Customers can see recommendations directly on the product page, which increases time spent on the site and improves overall user experience.

2.Question

How can teams work in parallel to implement Ajax integrations?

Answer:Teams can work in parallel by establishing a clear protocol for interaction, such as defining endpoint URLs for fragments. In this chapter, Team Inspire and Team Decide

More Free Book
Scan to Download
Listen It

agree on a new endpoint (`http://localhost:3002/fragment/recommendations/<sku>`) for loading fragment markup, allowing them to independently develop their respective components.

3.Question

What are the challenges that arise from using iframes for integrating content?

Answer:Iframes can lead to poor SEO rankings due to their complex semantic structure. They also present challenges in terms of usability, layout flexibility, and can complicate styling due to potential conflicts with other styles present on the page.

4.Question

Why is it important to configure a shared web server when working with micro frontends?

Answer:Using a shared web server, such as Nginx, allows multiple applications to be served from a single domain, simplifying resource management, reducing cross-domain issues, and enhancing overall application performance.

More Free Book
Scan to Download
Listen It

5.Question

What steps are necessary for Team Inspire to expose their recommendations as a fragment?

Answer:Team Inspire needs to create a new HTML file for the fragment that serves a streamlined version of the recommendations, set up its own CSS file for styling, and ensure proper server configuration for loading this fragment via Ajax.

6.Question

Can you illustrate the process of loading a fragment into the DOM using Ajax?

Answer:To load a fragment via Ajax, Team Decide retrieves the URL of the fragment stored in a data attribute of an element. Using the Fetch API, they send a request to this URL, receive the HTML content, and insert it into the web page's DOM. This seamless integration enhances page interactivity without the drawbacks of iframes.

7.Question

What is a potential issue with referencing CSS when integrating multiple fragments?

More Free Book
Scan to Download
Listen It

Answer:Referencing multiple CSS files in different fragments can lead to redundant link tags, making the page heavier and potentially causing style conflicts. It's important to streamline these references to optimize performance, which will be discussed in more advanced techniques in Chapter 10.

8.Question

What role does SEO play in the decision to shift from iframe integration to Ajax?

Answer:SEO is crucial for discovering new customers; thus, addressing its challenges is essential. The decision to move away from iframes to Ajax was heavily influenced by the need for better semantic structure, which improves search engine crawling and ranking capabilities.

9.Question

What example is given for using JavaScript to load content via Ajax?

Answer:An example provided illustrates how Team Decide uses JavaScript to query the DOM for the specific element to

More Free Book
Scan to Download
Listen It

insert the recommendations, retrieves the fragment URL from a data attribute, and then uses the Fetch API to load the HTML and append it to the product page.

10.Question

How does proper naming and directory structure aid in the integration process?

Answer:A proper directory structure clearly separates fragments and associated styles, enabling easy management and understanding of the codebase. This organization facilitates collaboration between teams by providing clarity on where specific files are located and their purpose.

Chapter 6 | 4 Server-side composition| Q&A

1.Question

What is the main advantage of using server-side composition for micro-frontends?

Answer:The main advantage of server-side composition is that the page is fully assembled before it reaches the customer's browser, leading to significantly improved first-page load speeds. This

More Free Book
Scan to Download
Listen It

approach enables better overall performance compared to client-side integrations, which often experience delays due to additional requests.

2.Question

How does Nginx handle server-side includes (SSI)?

Answer:Nginx processes SSI directives by replacing them with the content fetched from the specified URL before sending the final markup to the client. This process involves resolving includes in parallel, which speeds up the overall page response time.

3.Question

What challenges might arise if a fragment service experiences slow response times?

Answer:If a fragment service is slow to respond, it can significantly prolong the overall response time for the complete page, as server-side integration waits for all required fragments to load. This could ultimately degrade the user experience by making the application feel sluggish.

4.Question

Why are timeouts and fallbacks important in server-side

More Free Book
Scan to Download
Listen It

integrations?

Answer:Timeouts and fallbacks are important because they ensure that an error in one fragment does not cause the entire page to fail. They help maintain robustness by allowing the page to load successfully, even if some fragments are not available, thus avoiding a complete breakdown.

5.Question

What performance improvements can be achieved through parallel loading of fragments in server-side composition?

Answer:Using parallel loading allows Nginx to fetch multiple fragments simultaneously rather than sequentially. This significantly reduces the time it takes to generate the complete markup for the page, enhancing the speed at which the page becomes interactive for users.

6.Question

What strategies can be employed to optimize page load times when integrating multiple fragments?

Answer:To optimize page load times, consider employing

More Free Book
Scan to Download
Listen It

techniques such as deferring non-essential fragments to load asynchronously via Ajax calls after the main content is rendered. Additionally, implementing partial sending or streaming responses can allow the browser to start rendering parts of the page before all fragments are fully loaded.

7.Question

How can developers test the performance of a server-side integrated page under suboptimal network conditions?

Answer:Developers can simulate suboptimal network conditions by using browser developer tools to throttle network speeds, allowing them to observe how the page performs under limitations similar to those experienced by users on slower connections.

8.Question

What is the role of Edge-Side Includes (ESI) in server-side composition?

Answer:ESI offers a standardized way to handle markup assembly at the edge of a content delivery network (CDN), allowing for efficient inclusion of dynamic content while

More Free Book
Scan to Download
Listen It

handling fallbacks and timeouts, similar to SSI but with additional capabilities tailored for edge caching services.

9.Question

How does the streaming model enhance performance compared to traditional server-side methods?

Answer:The streaming model allows a composition service to start sending parts of the page to the client as they are ready rather than waiting for all fragments to load. This overlap of loading the main page and its fragments greatly improves time to first byte (TTFB) and overall perceived performance for the user.

10.Question

In what scenarios is lazy-loading of fragments recommended?

Answer:Lazy-loading is recommended for optional fragments that are lower down the page or not crucial for the core functionality, such as promotional banners. This ensures that essential content is prioritized and displays more quickly to enhance user experience.

More Free Book
Scan to Download
Listen It

Ad

Scan to Download

Read, Share, Empower

Finish Your Reading Challenge, Donate Books to African Children.

The Concept

BOOKS FOR AFRICA × ×

This book donation activity is rolling out together with Books For Africa. We release this project because we share the same belief as BFA: For many children in Africa, the gift of books truly is a gift of hope.

The Rule

Earn 100 points → **Redeem a book** → **Donate to Africa**

Your learning not only brings knowledge but also allows you to earn points for charitable causes! For every 100 points you earn, a book will be donated to Africa.

Free Trial with Bookey

Chapter 7 | 5 Client-side composition| Q&A

1.Question

What is the primary goal of using Web Components in micro frontends?

Answer:Web Components serve as a technology-agnostic glue that enables different microfrontends developed with various frameworks to coexist seamlessly on a single page, allowing for independent upgrades and a more flexible architecture.

2.Question

How does client-side rendering improve user experience compared to traditional server-side rendering?

Answer:Client-side rendering produces and updates HTML directly in the browser, making the application feel more interactive and responsive. Users experience faster interactions, as they don't have to wait for the entire page to reload with every input.

3.Question

What is the significance of Shadow DOM when

More Free Book
Scan to Download
Listen It

integrating micro frontends?

Answer:Shadow DOM provides a way to isolate a subtree of the DOM, preventing style conflicts and ensuring that the CSS defined for one component does not affect others. This enhances the robustness of micro frontend applications by guaranteeing style encapsulation.

4.Question

How does Team Checkout manage its integration with Team Decide in the example provided?

Answer:Team Checkout provides Team Decide with a standalone Buy button component as a Web Component, allowing Team Decide to seamlessly integrate it into their product page without needing to understand the internal workings of the checkout logic.

5.Question

What are the benefits and drawbacks of using Web Components for client-side integration?

Answer:Benefits include strong encapsulation features, wide standardization across browsers, and improved isolation of

More Free Book
Scan to Download
Listen It

styles. Drawbacks involve the requirement for client-side JavaScript, potential issues with server-side rendering, and heavier polyfills for older browser support.

6.Question

When should a team consider using Shadow DOM for their micro frontends?

Answer:Teams should consider using Shadow DOM when building interactive applications that require strong style isolation, especially when working with legacy applications where non-conflicting styling is critical. It is less suitable for simpler use cases where the overhead might not be justified.

7.Question

What is the role of lifecycle methods introduced by Custom Elements in micro frontends?

Answer:Lifecycle methods such as connectedCallback and disconnectedCallback allow teams to manage the initialization, updating, and deinitialization of their individual micro frontends, creating a standardized way to interact with and control the component.

More Free Book
Scan to Download
Listen It

8.Question

How does the use of Web Components impact the release cycles of micro frontends?

Answer:By using Web Components, teams can work independently on their frontend applications, leading to a decoupled architecture. This reduces the need for coordinated release cycles, allowing for faster iterations and updates without needing to rewrite the entire frontend.

9.Question

What considerations should be taken into account when deciding between client-side rendering and server-side rendering for micro frontends?

Answer:The choice depends on the application's interactivity needs. For highly interactive applications, client-side rendering might be preferable due to its responsiveness. For simpler content-heavy sites, server-side rendering could offer faster initial load times and better SEO.

Chapter 8 | 6 Communication patterns| Q&A

1.Question

What is the primary focus of the user interface

More Free Book
Scan to Download
Listen It

communication patterns in micro frontends?

Answer:The primary focus is on how different micro frontends communicate and update each other when events occur, such as when a user adds an item to a basket, thereby triggering updates across interconnected user interfaces.

2.Question

Why is minimal UI communication between teams encouraged in micro frontend architecture?

Answer:Minimal UI communication is encouraged to keep teams focused and allow users to interact primarily with one team's user interface at a time, resulting in a clearer and less convoluted customer journey.

3.Question

How do parent-to-child communications function in micro frontends?

Answer:Parent-to-child communication involves passing contextual information down from a parent UI component to a child component through attributes, enabling the child to

More Free Book
Scan to Download
Listen It

react to changes such as selections made by the user.

4.Question

What role do events play in fragment-to-parent communication?

Answer:Events allow a child component to notify its parent about actions or changes that have occurred, enabling the parent to react accordingly without tight coupling between components.

5.Question

What is the significance of using a global event bus for fragment-to-fragment communication?

Answer:A global event bus reduces coupling between fragments by allowing them to communicate without knowing about each other's internal workings. It enables one fragment to emit events and others to listen and react, fostering a more modular architecture.

6.Question

Can you explain how the addition of a platinum upgrade option demonstrates effective UI communication?

Answer:The addition of the platinum upgrade option

More Free Book
Scan to Download
Listen It

showcases UI communication as Team Decide updates the product page and communicates changes to Team Checkout's Buy button via an extended attribute, which reflects the updated edition in real-time.

7.Question

What are the benefits of utilizing the browser's native Custom Events API for communication between fragments?

Answer:Using the Custom Events API allows for clear, high-level event names, easy debuggability, and access to native event features, enabling fragments to communicate without needing to know about each other's implementation.

8.Question

How is the average cart size impacted by the replacement of traditional alerts with animations?

Answer:Replacing alerts with animations created a friendlier user experience, leading to measurable positive effects on user engagement and a 31% increase in average cart size, demonstrating the effectiveness of better UI communication.

9.Question

More Free Book
Scan to Download
Listen It

What considerations should developers keep in mind when exchanging data structures through events?

Answer:Developers should minimize payloads in events to reduce coupling, focusing primarily on notifications rather than data transfer. This helps maintain the modularity and independence of different fragments.

10.Question

How does the event-bus approach facilitate better team collaboration in micro frontends?

Answer:The event-bus approach facilitates collaboration by creating a clear communication channel where teams can define and listen to relevant events, allowing for flexible interaction and updates without needing direct access to each other's components.

Chapter 9 | 7 Client-side routing and the application shell| Q&A

1.Question

What is the main purpose of an application shell in a micro frontend architecture?

Answer:The application shell acts as a central

More Free Book
Scan to Download
Listen It

router for all incoming requests in a micro frontend system. It maps URLs to the appropriate micro frontends, ensuring that when a user navigates to a certain URL, the relevant page is rendered within the shared HTML structure of the application shell.

2.Question

How does soft navigation improve user experience in single-page applications?

Answer:Soft navigation allows users to switch between different pages of an application without full page refreshes, leading to a snappier experience. The browser only updates the parts of the page that need to change, making the application feel faster and more responsive.

3.Question

What are the two main types of navigation described in the chapter, and how do they differ?

Answer:The two main types of navigation are hard navigation and soft navigation. Hard navigation requires the browser to load a complete HTML document from the server

More Free Book
Scan to Download
Listen It

for the next page, resulting in a full page refresh. In contrast, soft navigation occurs entirely on the client side, allowing for transition between pages without refreshing the whole page.

4.Question

Why is it beneficial for teams to maintain independence in a micro frontend setup?

Answer:Maintaining independence allows teams to develop, deploy, and upgrade their sections of the application independently without causing disruptions to other teams, leading to faster iterations and better flexibility in adopting new technologies or responding to user needs.

5.Question

How can integrating an application shell simplify routing for multiple teams?

Answer:An application shell centralizes routing by defining a single point where requests are handled, allowing for client-side routing without requiring each team to implement its own routing solution. This reduces complexity and maintains a cohesive user experience across different micro

More Free Book
Scan to Download
Listen It

frontends.

6.Question

What considerations should teams keep in mind when implementing page transitions in a micro frontend architecture?

Answer:Teams should consider how they balance user experience with technical integration. The number of teams involved and the need for seamless navigation can impact decisions on whether to implement hard or soft navigations. Additionally, establishing clear contracts through URLs between teams can aid in smoother transitions.

7.Question

What are some of the challenges associated with building a technology-agnostic client-side router?

Answer:Building a technology-agnostic client-side router poses challenges in ensuring compatibility across different frameworks and versions used by teams. It requires careful planning and prototyping to avoid increased coupling while still enabling efficient communication and routing functionalities among micro frontends.

More Free Book
Scan to Download
Listen It

8.Question

Can you explain the difference between the 'links only' model and the 'linked single-page apps' model?

Answer:In the 'links only' model, all navigation happens through plain links resulting in full page refreshes for every transition, regardless of team boundaries. In the 'linked single-page apps' model, transitions within the same team are handled via soft navigation, but crossing into another team's domain triggers hard navigation.

9.Question

What is a core benefit of the architecture used by Tractor Models, Inc., as described in the chapter?

Answer:One core benefit of the architecture is its ability to facilitate rapid development and deployment of the online shop's features due to the independence of teams working on their own micro frontends. This structure promotes agility and a better customer experience while allowing for continuous evolution of the application.

10.Question

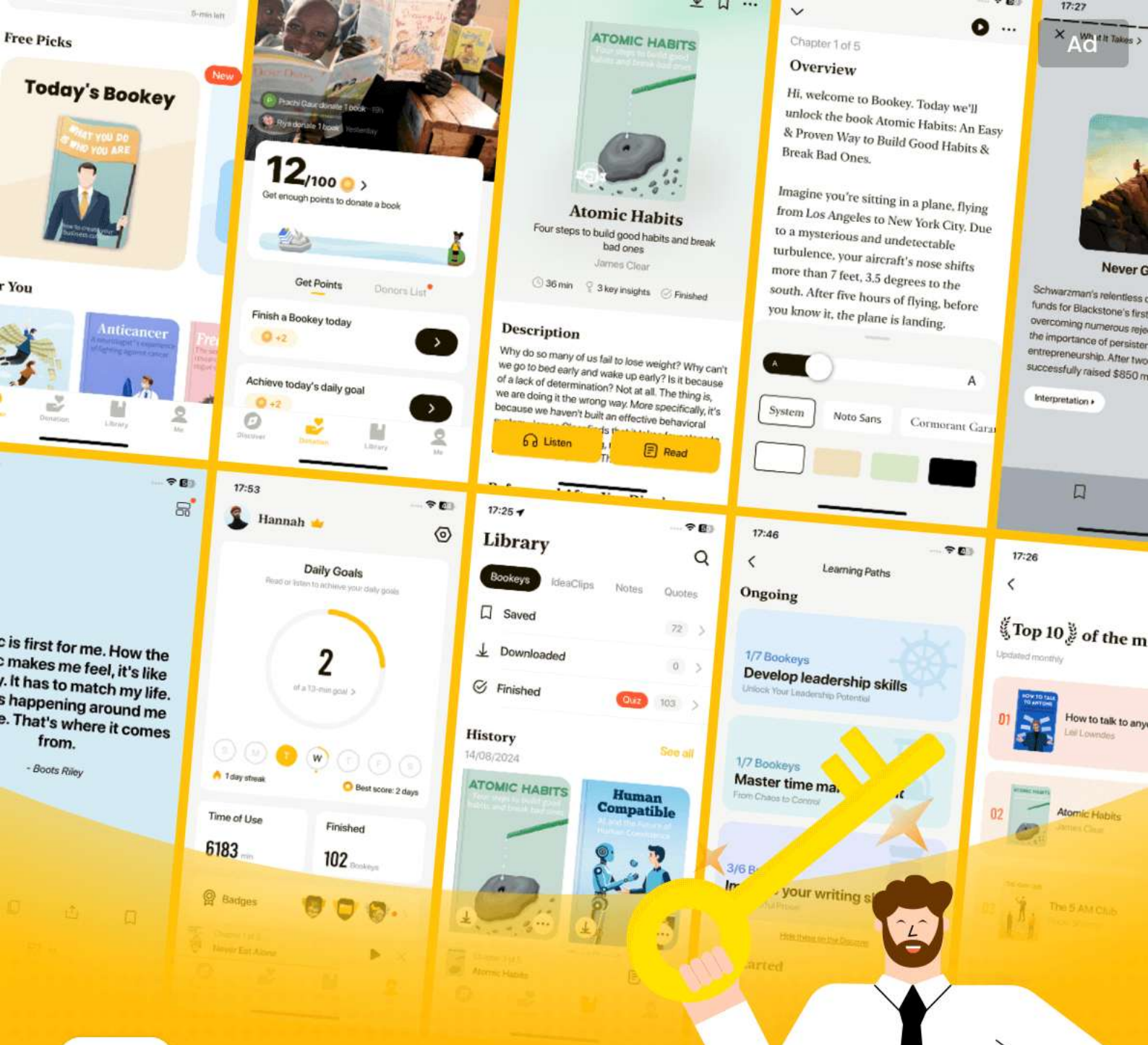
How does the chapter suggest building an application

More Free Book
Scan to Download
Listen It

shell, and what are its key components?

Answer:The chapter suggests building the application shell from scratch, emphasizing simplicity and minimalism, ensuring it does not contain business logic. Its key components include providing a shared HTML document, mapping URLs to team pages (client-side routing), rendering matching pages, and properly initializing or de-initializing components during navigation.

More Free Book
Scan to Download
Listen It



World' best ideas unlock your potential

Free Trial with Bookey

Scan to download

Download on the App Store

GET IT ON Google Play

Chapter 10 | 8 Composition and universal rendering| Q&A

1.Question

What is universal rendering, and why is it significant in micro frontends architecture?

Answer:Universal rendering, also referred to as isomorphic JavaScript or server-side rendering (SSR), enables a single codebase to render markup both on the server and in the browser. This technique is significant because it combines the fast initial load times of server-side rendering with the rich interactivity of client-side applications, ensuring a smoother user experience.

2.Question

How can server-side and client-side composition techniques work together effectively?

Answer:Server-side composition can be utilized to assemble the initial HTML document quickly using techniques like Server Side Includes (SSI), while client-side composition enables individual micro frontends to hydrate and become

More Free Book
Scan to Download
Listen It

interactive after the initial page load. This tandem approach ensures users receive content swiftly and allows for dynamic interactions without noticeable lag.

3.Question

What challenges might developers face when implementing universal composition?

Answer:The complexity of setting up, running, and debugging universal applications increases compared to solely server-side or client-side approaches. Developers must understand both server-side integration and client-side hydration, which requires more extensive training and can complicate the onboarding process for new developers.

4.Question

Why is progressive enhancement emphasized in the context of universal rendering?

Answer:Progressive enhancement is crucial in ensuring that core functionalities remain accessible even when JavaScript fails to load. By building applications that work without JavaScript, developers provide a failsafe experience,

More Free Book
Scan to Download
Listen It

allowing users to accomplish essential tasks regardless of their environment.

5.Question

What are best practices for maintaining contracts between teams in a micro frontends architecture?

Answer:Establish a standardized naming schema for micro frontends and their endpoints to reduce redundancy and friction. Each team should outline the necessary components they provide, such as custom elements for rendering and server-side endpoints for markup delivery, ensuring clear communication and successful integration.

6.Question

When is it necessary to adopt universal composition techniques?

Answer:Universal composition techniques become essential when the application requires fast first-page loads and high levels of interactivity. If micro frontends need to communicate constantly or are crucial to the user experience, a hybrid model combining both server and client rendering is

More Free Book
Scan to Download
Listen It

beneficial.

7.Question

What is the future outlook on creating a universal unifying single-page application (SPA)?

Answer:While theoretically feasible, combining single-page application principles with universal rendering introduces substantial complexity. The ongoing development of frameworks like single-spa shows promise in this area, but practical implementations remain limited as of now.

8.Question

How can the challenges of micro frontends be mitigated without compromising performance?

Answer:By leveraging techniques such as universal rendering to combine benefits of server and client rendering, and focusing on progressive enhancement, teams can mitigate the common challenges of maintaining micro frontends while providing a robust and responsive user experience.

9.Question

What role do frameworks play in easing the implementation of universal applications?

More Free Book
Scan to Download
Listen It

Answer:Modern JavaScript frameworks significantly ease the development of universal applications by providing built-in support for SSR and hydration, allowing developers to focus more on feature implementation rather than the complexities of rendering across environments.

10.Question

How does universal rendering optimize user experience in a micro frontends structure?

Answer:Universal rendering optimizes user experience by delivering fast initial loads and responsive interactions, allowing users to access content quickly while enabling interactivity that enhances engagement with the application.

Chapter 11 | 9 Which architecture fits my project?| Q&A

1.Question

What are the key integration techniques for micro frontends discussed in this chapter?

Answer:The chapter covers several key integration techniques for micro frontends, including links, iframes, Ajax, server-side integration, Web

More Free Book
Scan to Download
Listen It

Components, and the app shell model. Each technique offers a different approach to integrating user interfaces owned by different teams.

2.Question

How does the Documents-to-Applications Continuum help in deciding the architecture for a micro frontend project?

Answer:The Documents-to-Applications Continuum serves as a mental model that aids in determining whether a project is more content-centric or behavior-centric. By positioning the project on this continuum, teams can better understand the right priorities and suitable architectures for their specific needs.

3.Question

What are the advantages and challenges of using linked pages as a micro frontend architecture?

Answer:The linked pages architecture is simple to implement and requires no centralized infrastructure or shared code, making debugging straightforward. However, it can lead to slower navigation and a less seamless user experience as it

More Free Book
Scan to Download
Listen It

relies on full page reloads.

4.Question

In what scenarios might a Unified Universal SPA be the best architecture choice?

Answer:A Unified Universal SPA is best suited for applications that require high interactivity and fast initial rendering. If your project combines heavy content and complex user interactions, adopting this architecture can provide a snappier and more cohesive user experience.

5.Question

What considerations should teams make when deciding between server-side and client-side integration techniques?

Answer:Teams should evaluate whether their product has a strong focus on content (favoring server-side) or interaction (favoring client-side). If the project lies in between, they may need to consider the trade-offs of both techniques and potentially adopt universal rendering if they require the benefits of both server and client-side integration.

6.Question

More Free Book
Scan to Download
Listen It

Why is it important to establish a shared vocabulary among teams working on a micro frontend project?

Answer:A shared vocabulary is crucial for avoiding misunderstandings among teams about the integration techniques and architectures being used. It ensures that all members have a clear understanding of the project, which facilitates better collaboration and streamlined development.

7.Question

What role does the architecture decision guide play in the micro frontend development process?

Answer:The architecture decision guide helps teams make informed choices about the appropriate architecture and integration technique for their project based on specific criteria and questions that pertain to the project's goals, user interactions, and the level of integration required.

8.Question

How can strong technical isolation be prioritized when integrating legacy systems into a micro frontend architecture?

Answer:Strong technical isolation should be prioritized if

More Free Book
Scan to Download
Listen It

integrating a legacy system that does not respect namespacing rules or requires global state. It is necessary in cases where security is a concern, such as when dealing with untrusted third-party systems.

9.Question

How does the concept of progressive enhancement relate to web application performance?

Answer:Progressive enhancement is vital for ensuring good web application performance, particularly for content-centric projects. This approach allows the core content to be accessible without relying on advanced features, which can improve load times and overall user satisfaction.

10.Question

What is the significance of soft navigation in a micro frontends architecture?

Answer:Soft navigation enhances the user experience by allowing for seamless transitions between different parts of an application without full page reloads. This is particularly important for applications where users frequently switch

More Free Book
Scan to Download
Listen It

contexts, as it makes interactions feel more fluid and responsive.

Chapter 12 | Part 3. How to be fast, consistent, and effective| Q&A

1.Question

What are some key focuses in ensuring a successful micro frontends project?

Answer:Beyond just integration, focus on performance, visual consistency, and team responsibility. These aspects will be further elaborated in subsequent chapters.

2.Question

Why is asset loading considered a technical challenge in micro frontends?

Answer:Loading the right scripts and styles for different micro frontends can be complex, especially when trying to maintain performance best practices while enabling team autonomy.

3.Question

What common pitfalls are associated with performance in

More Free Book
Scan to Download
Listen It

micro frontends?

Answer:Common pitfalls include inefficient loading strategies and failure to optimize resources, which can lead to slow site performance. Strategies will be discussed to help avoid these issues.

4.Question

Why is a shared design system necessary in a micro frontends architecture?

Answer:A shared design system helps maintain a consistent look and feel across different micro frontends, ensuring that end users have a unified experience.

5.Question

What challenges arise when establishing a shared design system in micro frontends?

Answer:The main challenge is balancing the need for a consistent UX with the autonomy of individual teams to innovate and make decisions about their own frontends.

6.Question

What organizational implications should teams consider when adopting micro frontends?

More Free Book
Scan to Download
Listen It

Answer:Teams should consider how to define clear team boundaries, facilitate knowledge transfer between teams, and manage any shared infrastructure effectively.

7.Question

What key topics will be covered in Chapter 13 regarding micro frontends?

Answer:Chapter 13 will focus on team boundaries, inter-team knowledge transfer, and dealing with shared infrastructure when using micro frontends.

8.Question

What are some best practices for local development and testing in a micro frontends approach?

Answer:While specific practices will be addressed in Chapter 14, they may include setting up isolated environments for individual micro frontends and utilizing effective testing patterns.

More Free Book
Scan to Download
Listen It

Ad

Scan to Download

Try Bookey App to read 1000+ summary of world best books

Unlock 1000+ Titles, 80+ Topics

New titles added every week

Brand | Leadership & Collaboration | Time Management | Relationship & Communication

ness Strategy | Creativity | Public | Money & Investing | Know Yourself | Positive P

Entrepreneurship | World History | Parent-Child Communication | Self-care | Mind & Spi

Insights of world best books

Free Trial with Bookey

Chapter 13 | 10 Asset loading| Q&A

1.Question

What are the key challenges associated with loading assets in a micro frontends architecture?

Answer:The key challenges include ensuring that teams can independently deploy their micro frontends and the required assets, implementing effective cache busting without creating tight coupling, maintaining alignment between loaded CSS and JS with the server-generated markup, and deciding on an appropriate bundling strategy—whether to use many smaller bundles or fewer larger ones.

2.Question

How does direct referencing work in the context of micro frontends?

Answer:Direct referencing involves explicitly including the necessary asset links, such as JavaScript and CSS files, for each micro frontend into the integrating application's page.

More Free Book
Scan to Download
Listen It

This is akin to adding import statements in programming languages and ensures that all required assets are loaded. Each team is responsible for publishing their asset URLs, which the integrating team then includes in their HTML.

3.Question

What advantages does on-demand loading provide in a micro frontends setup?

Answer:On-demand loading allows assets to be loaded dynamically when needed rather than upfront, which can significantly reduce the initial load time and improve performance. This technique is part of frameworks like single-spa, where assets are only fetched when a user interacts with a part of the application that requires them, thus optimizing resource usage and providing a smoother user experience.

4.Question

What is the best practice for including stylesheets and scripts in a micro frontends application?

Answer:To ensure fast rendering, it is best practice to include

More Free Book
Scan to Download
Listen It

stylesheets in the <head> of the HTML document and to load script tags asynchronously at the end of the <body>. This approach minimizes render-blocking and improves the overall load performance of the application.

5.Question

Why is it important to consider the granularity of bundles in micro frontends?

Answer:Choosing the right granularity for bundles is crucial as it impacts the loading speed, manageability, and maintainability of the application. Smaller bundles allow for faster loading of only what is needed, while larger bundles can simplify deployment but may lead to longer load times. The decision should align with user experience goals and team structure.

6.Question

What role does cacheability play in asset loading for micro frontends?

Answer:Cacheability is essential in optimizing load times and reducing server demand by allowing browsers to store

More Free Book
Scan to Download
Listen It

and reuse previously fetched files. Implementing cache busting strategies that do not introduce tight coupling between teams is necessary to ensure that updates to assets are recognized without affecting the independence of individual micro frontends.

7.Question

How should teams coordinate the sharing of assets in a micro frontends architecture?

Answer:Teams should provide documentation that includes the URLs for their respective assets, along with any necessary initialization code and styling instructions. Clear communication and well-defined interfaces ensure that integrating teams can effectively use the micro frontends without confusion or redundancy.

Chapter 14 | 11 Performance is key| Q&A

1.Question

What are the primary concerns developers have when first introduced to micro frontends?

Answer:Developers often express concerns about the

More Free Book
Scan to Download
Listen It

perceived inefficiency and overhead of having multiple teams create their own separate frontends. They worry that this approach might lead to slower performance compared to monolithic frontend architectures.

2.Question

How can performance be effectively measured in a micro frontends architecture?

Answer:Effective performance measurement can be established by defining performance budgets for different metrics across teams. For instance, teams can set a goal that the total weight of a page should not exceed 1MB, and track load times to ensure optimal user experience.

3.Question

What strategies are recommended for addressing performance bottlenecks in micro frontends?

Answer:To address performance bottlenecks, teams should focus on isolating the performance of their individual micro frontends, measuring their impact, and tracking regressions

More Free Book
Scan to Download
Listen It

using centralized metrics systems. Collaboration among teams to investigate performance issues is also crucial.

4.Question

Why is it important to set a 'definition of fast' for different parts of a micro frontend application?

Answer:Defining what 'fast' means for various parts of the application ensures that performance expectations align with user needs. For example, loading times for the homepage should be prioritized differently than those for a checkout process, where interactive speed is critical.

5.Question

What role does vendor library sharing play in improving performance?

Answer:By sharing large vendor libraries across teams, micro frontend projects can significantly reduce the amount of duplicated JavaScript that needs to be loaded, thus lowering network traffic and improving overall performance.

6.Question

How can autonomous teams maintain performance without compromising their independence?

More Free Book
Scan to Download
Listen It

Answer:Teams can maintain their independence by using a shared performance budget and agreeing upon common performance guidelines that respect individual team autonomy while ensuring that collaboration exists to uphold the overarching performance goals.

7.Question

What steps should a team take if their performance metrics exceed budget?

Answer:If performance metrics exceed budget, the team should halt new developments, investigate the causes of degradation, and collaboratively discuss options to realign with performance budgets, which may include rolling back changes or optimizing existing features.

8.Question

What is the significance of monitoring performance in micro frontends?

Answer:Monitoring is crucial in micro frontends as it allows teams to track performance in real time, enabling them to quickly identify and address issues as they arise. This

More Free Book
Scan to Download
Listen It

proactive approach helps maintain a high level of user satisfaction.

9.Question

How do small frameworks influence the performance of micro frontends?

Answer:Choosing smaller frameworks can reduce overhead since they introduce less bulk into the codebase, minimizing the risk of performance degradation due to redundant downloads and allowing faster load times for users.

10.Question

What benefits arise from the use of micro frontends despite potential performance challenges?

Answer:Micro frontends provide benefits such as built-in code splitting, optimized user experience through focused scope, and enhanced adaptability to specific use cases, all of which can lead to better overall performance.

Chapter 15 | 12 User interface and design system| Q&A

1.Question

What is the purpose of a design system in micro

More Free Book
Scan to Download
Listen It

frontends?

Answer:A design system in micro frontends serves to deliver a consistent user experience across different teams and components, allowing product teams to independently create and maintain their features while adhering to a unified design language. It ensures that components like color palettes, typography, and UI elements feel familiar to users, thereby enhancing usability and satisfaction.

2.Question

Why is it important to integrate a design system early in the development process?

Answer:Integrating a design system early is crucial because decisions about design architecture influence all user-facing features. If set in the early stages, it can effectively govern consistency and quality across components, and avoid costly revisions later in the development cycle.

3.Question

What are the trade-offs between a centralized design

More Free Book
Scan to Download
Listen It

system and autonomous teams?

Answer:A centralized design system can streamline consistency and organization, but it may become a bottleneck, delaying feature releases as teams depend on it for updates. In contrast, autonomous teams can move faster and innovate but risk inconsistency unless well-coordinated. Striking the right balance is key.

4.Question

What role does communication play in the success of a design system?

Answer:Effective communication is vital for the success of a design system as it facilitates collaboration between design system teams and product teams. Establishing clear channels ensures that all teams understand the components available, fostering strong relationships and reducing the tendency to circumvent the design system.

5.Question

How does a well-implemented design system contribute to scaling an application?

More Free Book
Scan to Download
Listen It

Answer:A well-implemented design system contributes to scaling by providing a shared foundation of components that new teams can build upon. This reduces redundancy of discussions about design choices and speeds up the development process, allowing for faster feature delivery across multiple teams.

6.Question

What factors should be considered to determine if a component should be included in the central pattern library or left with a team?

Answer:Factors include the component's complexity (simpler elements should be centralized), its reusability across teams, and whether it is domain-specific. Components needed by multiple teams or those that represent the core brand should typically be centralized, whereas highly specific features or complex components may be better managed by individual teams.

7.Question

Why is maintaining a design system an ongoing process?

More Free Book
Scan to Download
Listen It

Answer:Maintaining a design system is ongoing because user requirements evolve, new technologies emerge, and design trends change. Continuous feedback from product teams allows for timely refinements and enhancements, ensuring that the design system remains relevant, useful, and effective.

8.Question

What are the consequences of failing to update a design system?

Answer:Failing to update a design system can render it obsolete, leading to inconsistencies in the user interface and creating a 'zombie' design system that teams may circumvent. This reduces the overall quality of the user experience and can result in costly redesigns as components become outdated.

9.Question

In what ways can a federated model enhance a design system's effectiveness?

Answer:A federated model enhances a design system by empowering product teams to contribute directly to its

More Free Book
Scan to Download
Listen It

development, which fosters ownership and accountability. Designers embedded within product teams can provide immediate input based on real use cases, leading to a more responsive and dynamic design system that evolves in line with actual user needs.

10.Question

How can integrating feedback loops improve the functionality of a design system?

Answer:Integrating feedback loops allows teams to continuously evaluate and refine components based on user experiences and changing needs. This ensures that the system not only remains aligned with user expectations but also supports innovation by allowing teams to surface and address issues in real-time.

More Free Book
Scan to Download
Listen It

Ad

Scan to Download

Why Bookey is must have App for Book Lovers

30min Content
The deeper and clearer interpretation we provide, the better grasp of each title you have.

Text and Audio format
Absorb knowledge even in fragmented time.

Quiz
Check whether you have mastered what you just learned.

And more
Multiple Voices & fonts, Mind Map, Quotes, IdeaClips...

Free Trial with Bookey

Chapter 16 | 13 Teams and boundaries| Q&A

1.Question

What are the primary benefits of adopting a micro frontends architecture?

Answer:The primary benefits of adopting a micro frontends architecture lie in its organizational advantages rather than purely technical ones. It enables parallel development, fostering real team ownership and local decision-making, which can accelerate innovation and improve product alignment with user needs.

2.Question

How does Conway's Law inform the structuring of teams in software development?

Answer:Conway's Law suggests that the communication structures of an organization influence its design outcomes. Therefore, aligning team boundaries with application boundaries promotes more modular architecture, enhancing coherence and reducing bottlenecks.

More Free Book
Scan to Download
Listen It

3.Question

What strategies can teams use to identify effective boundaries for micro frontend applications?

Answer:Teams can use strategies such as Domain-Driven Design (DDD) to analyze expert languages and identify bounded contexts. Another technique involves examining the existing page structure to align teams with user needs and functionality.

4.Question

What role does culture play in the success of a micro frontends architecture?

Answer:A user-focused culture that promotes team autonomy and embraces agile values is essential for successfully implementing a micro frontends architecture. This culture encourages teams to take ownership, experiment, and swiftly adapt to changes.

5.Question

How can teams foster knowledge sharing without duplicating efforts?

Answer:Establishing communities of practice (CoPs) allows

More Free Book
Scan to Download
Listen It

teams to communicate and share insights on common challenges, minimizing the risk of reinventing the wheel. Regular guild meetings or collaborative platforms facilitate this exchange.

6.Question

What is the significance of full-stack teams in the micro frontends architecture?

Answer:Full-stack teams enhance the micro frontends architecture by integrating various disciplines—from frontend to backend to operations—allowing for end-to-end feature delivery and reducing dependencies on other teams, thereby speeding up the development process.

7.Question

How can organizations leverage technology diversity effectively within micro frontends?

Answer:Organizations can implement a 'toolbox' approach that provides vetted technology choices while allowing teams the freedom to deviate from defaults when justified. This balances uniformity and innovation.

More Free Book
Scan to Download
Listen It

8.Question

What considerations should guide the use of shared infrastructure among teams?

Answer:When deciding on shared infrastructure, teams should evaluate the necessity of shared services, whether they require specialized expertise, and ensure that such infrastructure does not impede team autonomy or create bottlenecks.

9.Question

How do rituals like presenting work contribute to team collaboration in a micro frontends environment?

Answer:Rituals such as regular presentations allow teams to showcase their work, creating awareness and opportunities for collaboration. This practice fosters a culture of transparency and shared learning across different teams.

10.Question

Why is it essential to maintain an open culture based on agile values in micro frontends architecture?

Answer:An open culture based on agile values encourages collaboration, quick decision-making, and experimentation,

More Free Book
Scan to Download
Listen It

which are critical for adapting and thriving in the fast-paced environments typical of micro frontends development.

Chapter 17 | 14 Migration, local development, and testing| Q&A

1.Question

What are the primary reasons for migrating to a micro frontends architecture?

Answer:Companies typically migrate to micro frontends due to challenges with maintaining a monolithic architecture as team size increases or as demand for features grows. Micro frontends offer better scalability, independence for teams, and the ability to innovate more quickly.

2.Question

Can you describe an effective migration strategy for transitioning from a monolith to micro frontends?

Answer:One effective strategy is the 'slice-by-slice' approach. Teams incrementally migrate features from the monolith to independent micro frontends while ensuring integration is maintained. This minimizes risk and allows for continual

More Free Book
Scan to Download
Listen It

production.

3.Question

What role does a proof of concept play in the migration to micro frontends?

Answer:A proof of concept can serve as a lighthouse project, demonstrating how a single feature can be successfully developed as a micro frontend. It provides a model for other teams to follow during the overall migration.

4.Question

How do teams manage local development in a micro frontends architecture?

Answer:In a micro frontends setup, each team works independently on their own codebase and avoids running code from other teams. They often use mock fragments to simulate other parts of the application, allowing them to focus on their development without external dependencies.

5.Question

What are some techniques for testing micro frontends?

Answer:Testing in a micro frontends architecture involves unit tests and isolated UI tests for individual fragments or

More Free Book
Scan to Download
Listen It

components. Teams also conduct integration tests where necessary, particularly at interfaces between different teams' micro frontends.

6.Question

Why is mocking fragments important in a micro frontends environment?

Answer:Mocking fragments allows developers to work on their micro frontends without dependencies on other teams' code, which helps to avoid integration issues and maintain a stable local development environment.

7.Question

What are the benefits and challenges of a greenfield approach when migrating to micro frontends?

Answer:The greenfield approach allows teams to start fresh without legacy code issues, speeding up development and allowing for modern practices. However, it comes with the risk of lengthy development cycles without user feedback, which can lead to misalignment with user needs.

8.Question

What is the testing pyramid, and how does it apply to

More Free Book
Scan to Download
Listen It

micro frontends?

Answer:The testing pyramid illustrates that lower-level tests (like unit tests) are cheaper and faster, while higher-level tests (like end-to-end) are slower and more expensive to maintain. In micro frontends, testing can be split into isolated tests within individual teams and integration tests across team boundaries.

9.Question

How can teams handle inter-team dependencies in a micro frontends architecture?

Answer:It is essential to maintain clear boundaries and minimize shared dependencies between teams. Using separate repositories and mock implementations can help reduce friction and allow teams to work more independently.

10.Question

What does the term 'brownfield project' refer to in the context of migrating to micro frontends?

Answer:A brownfield project refers to the incremental migration of a legacy system (like a monolith) to new

More Free Book
Scan to Download
Listen It

technologies (like micro frontends), where both old and new systems coexist and gradually improve over time.

More Free Book
Scan to Download
Listen It

Ad

Scan to Download

App Store
Editors' Choice

22k 5 star review

Positive feedback

Sara Scholz

tes after each book summary
erstanding but also make the
and engaging. Bookey has
ding for me.

Fantastic!!!

Masood El Toure

I'm amazed by the variety of books and languages Bookey supports. It's not just an app, it's a gateway to global knowledge. Plus, earning points for charity is a big plus!

Fi
Ab
bo
to
my

José Botín

ding habit
p's design
ual growth

Love it!

Wonnie Tappkx

Bookey offers me time to go through the important parts of a book. It also gives me enough idea whether or not I should purchase the whole book version or not! It is easy to use!

Time saver!

Bookey is my go-to app for
summaries are concise, ins
curated. It's like having acc
right at my fingertips!

Awesome app!

Rahul Malviya

I love audiobooks but don't always have time to listen to the entire book! bookey allows me to get a summary of the highlights of the book I'm interested in!!! What a great concept !!!highly recommended!

Beautiful App

Alex Walk

This app is a lifesaver for book lovers with busy schedules. The summaries are spot on, and the mind maps help reinforce wh I've learned. Highly recommend!

Free Trial with Bookey

Micro Frontends in Action Quiz and Test

Check the Correct Answer on Bookey Website

Chapter 1 | Micro Frontends in Action| Quiz and Test

1. The term 'micro frontends' was introduced by the ThoughtWorks Technology Radar in November 2016.
2. Geers believes monolithic architectures can adequately support independent feature development across teams.
3. The book 'Micro Frontends in Action' focuses heavily on specific tools and libraries for implementing micro frontends.

Chapter 2 | 1 What are micro frontends?| Quiz and Test

1. Micro frontends are suitable for both small and large projects without any downsides.
2. Micro frontends allow teams to select and upgrade their technology stacks independently.
3. The integration of micro frontends can only occur on the

More Free Book
Scan to Download
Listen It

client-side, as server-side integration is not applicable.

Chapter 3 | 2 My first micro frontends project| Quiz and Test

1. Micro frontends allow multiple teams to work independently on complex applications.
2. The Tractor Store project uses a single technology stack for both teams to ensure uniformity.
3. The best integration approach for micro frontends is the same for every use case.

More Free Book
Scan to Download
Listen It

Scan to Download

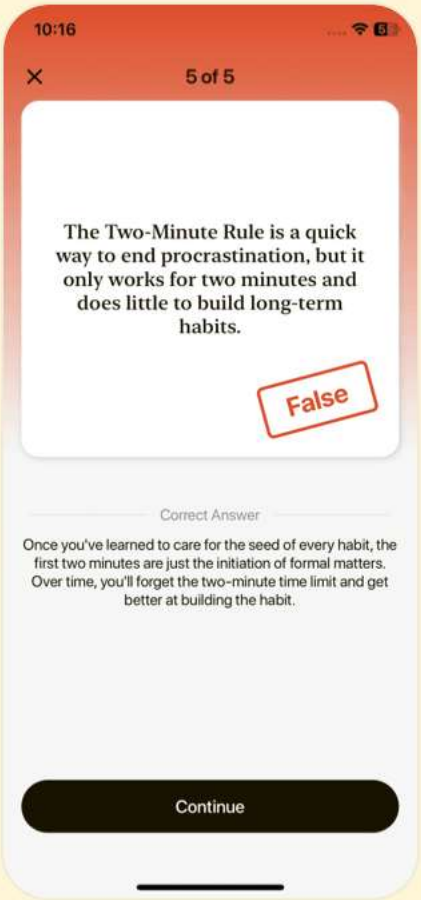
Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Download on the App Store

GET IT ON Google Play



Chapter 4 | Part 2. Routing, composition, and communication| Quiz and Test

1. The adoption of micro frontends is broadening within the software industry leading to a decrease in the availability of meta-frameworks and helper libraries.
2. Building advanced micro frontends projects will rely heavily on specialized tools.
3. A strong understanding of core concepts is essential for successfully building micro frontends projects.

Chapter 5 | 3 Composition with Ajax and server-side routing| Quiz and Test

1. This chapter covers integrating fragments into a page via Ajax for better SEO results.
2. The chapter suggests using iframes for fragment integration to enhance usability.
3. Team Inspire is responsible for delivering recommendations as HTML files that include their own CSS for styling.

More Free Book
Scan to Download
Listen It

Chapter 6 | 4 Server-side composition| Quiz and Test

1. Server-side composition assembles various fragments into a complete page on the client before sending it to the user.
2. Nginx's Server-Side Includes (SSI) can improve load times by allowing the inclusion of asset URLs directly in HTML responses.
3. Implementing timeouts for server-side integrations can stop unresponsive fragments from hindering overall page performance.

More Free Book
Scan to Download
Listen It

Scan to Download

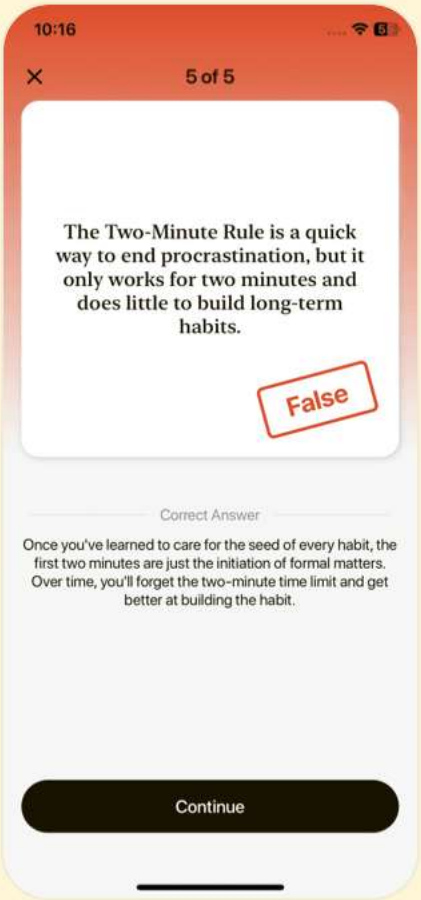
Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Download on the App Store

GET IT ON Google Play



Chapter 7 | 5 Client-side composition| Quiz and Test

1. Micro frontends can be integrated on the same page using different frameworks without conflicts due to the use of Web Components.
2. Shadow DOM is used to allow styles from one micro frontend to affect another micro frontend on the same page.
3. Web Components are beneficial for creating reusable components that can function like standard HTML elements, but they can also present challenges with older browser compatibility.

Chapter 8 | 6 Communication patterns| Quiz and Test

1. UI components from different teams need to communicate when users interact with various elements.
2. State management in micro frontends should prioritize duplication over consistency across components.
3. Communication between fragments can enhance coupling through direct dependencies.

More Free Book
Scan to Download
Listen It

Chapter 9 | 7 Client-side routing and the application shell| Quiz and Test

1. The application shell is responsible for managing incoming requests and mapping URLs to the correct micro-frontend for rendering.
2. The Links Only Model results in soft navigation between team boundaries.
3. An application shell should incorporate complex business logic to ensure effective management of micro frontends.

More Free Book
Scan to Download
Listen It

Scan to Download

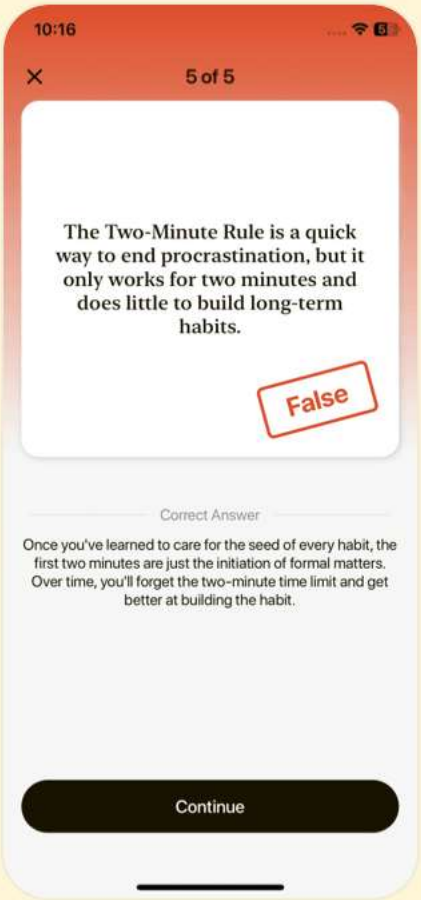
Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Download on the App Store

GET IT ON Google Play



Chapter 10 | 8 Composition and universal rendering| Quiz and Test

1. Universal rendering allows for the rendering of markup using a single codebase on both the server and client.
2. Adopting a hybrid integration model using only client-side technologies will remedy quality issues caused by client-side JavaScript errors.
3. Establishing a clear contract for integration between teams helps in streamlining micro frontends integration.

Chapter 11 | 9 Which architecture fits my project?| Quiz and Test

1. Establishing a common vocabulary across teams is essential when setting up a micro frontends project.
2. The Documents-to-Applications Continuum only applies to content-centric projects.
3. Different teams can adopt varied architectures leading to flexibility but complicating integration.

More Free Book
Scan to Download
Listen It

Chapter 12 | Part 3. How to be fast, consistent, and effective| Quiz and Test

1. Loading the right script and style code for different micro frontends is straightforward and easy.
2. Design systems are crucial for maintaining a consistent look and feel across micro frontends.
3. Establishing shared design systems does not interfere with the autonomy goals of micro frontends.

More Free Book
Scan to Download
Listen It

Scan to Download

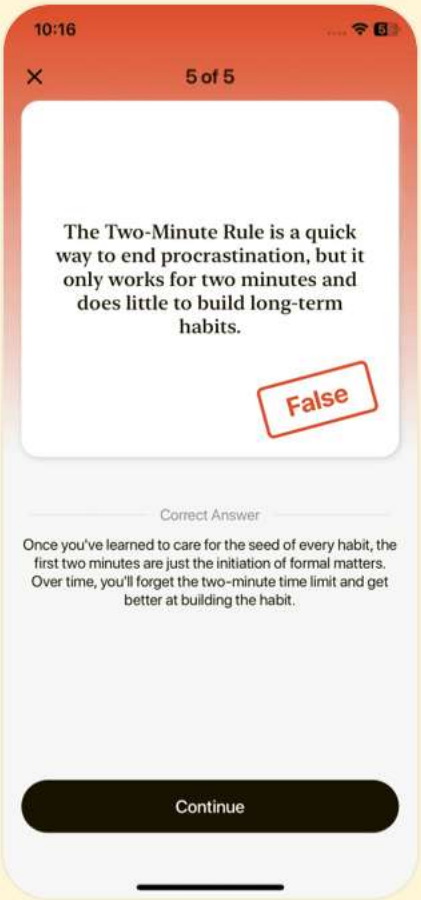
Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Download on the App Store

GET IT ON Google Play



Chapter 13 | 10 Asset loading| Quiz and Test

1. Micro frontends allow teams to deploy their assets independently without tight coupling.
2. Using direct referencing, teams can only reference assets from their own micro frontends.
3. Preloading all assets from all teams is always the most efficient method for loading assets in a micro frontend architecture.

Chapter 14 | 11 Performance is key| Quiz and Test

1. Performance budgets help establish a culture focused on performance by encouraging ongoing measurement with tools like Lighthouse.
2. A performance-first approach involves allowing page weight to exceed 1MB to accommodate larger micro frontends.
3. Utilizing a global script for shared frameworks does not improve load efficiency in micro frontends.

Chapter 15 | 12 User interface and design system| Quiz and Test

More Free Book
Scan to Download
Listen It

1. A design system can help create a consistent user experience across various micro frontend teams.
2. Building a design system is an easy and quick process that does not require much planning or investment.
3. Centralizing all components in a pattern library is always the best practice for maintaining team autonomy.

More Free Book
Scan to Download
Listen It

Scan to Download

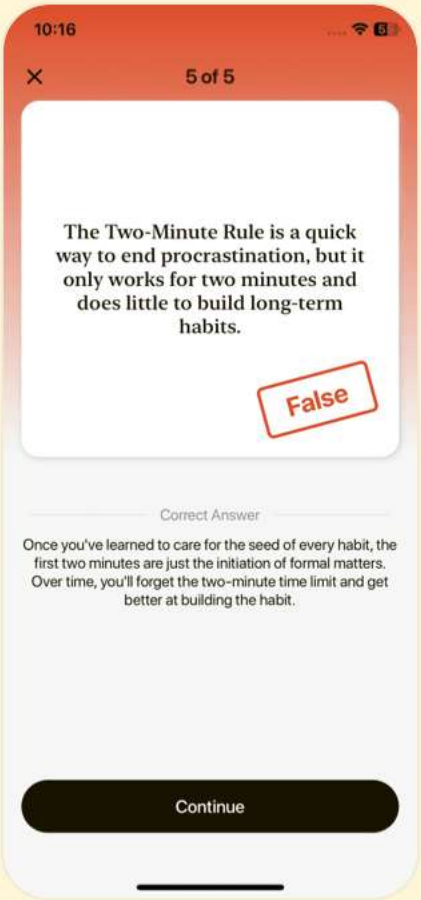
Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Download on the App Store

GET IT ON Google Play



Chapter 16 | 13 Teams and boundaries| Quiz and Test

1. Teams should be structured based on technology diversity rather than user needs.
2. Micro frontend teams should ideally encompass the entire stack to enhance decision-making autonomy.
3. A culture based on agile values is not important for the success of micro frontend architectures.

Chapter 17 | 14 Migration, local development, and testing| Quiz and Test

1. Migrating a monolithic application to a micro frontends architecture is a simple process that requires minimal effort.
2. The slice-by-slice migration strategy allows for incremental migration while keeping the application functional.
3. In micro frontends development, teams should run code from other teams to manage dependencies effectively.

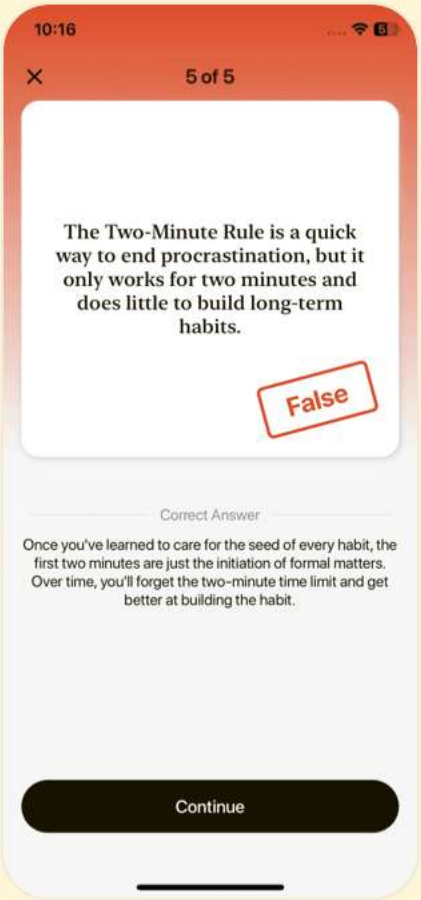
More Free Book
Scan to Download
Listen It

Scan to Download

Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Download on the App Store

GET IT ON Google Play